



ETSI GS QKD 020 V1.1.1 (2026-06)

그룹 규격(GROUP SPECIFICATION)

양자 키 분배(QKD); REST 기반 상호운용 키 관리 시스템 API의 프로토콜 및 데이터 형식

(Quantum Key Distribution (QKD); Protocol and data format of REST-based Interoperable Key Management System API)

면책 조항(Disclaimer)

이 문서는 양자 키 분배(QKD) ETSI 산업 규격 그룹(ISG)이 작성하고 승인하였으며, 이 ISG에 참여한 회원들의 견해를 나타낸다. 이 문서가 반드시 ETSI 전체 회원의 견해를 나타내는 것은 아니다.

참조 번호(Reference)

DGS/QKD-020_InteropKMS

키워드(Keywords)

API, protocol, quantum cryptography, quantum key distribution

ETSI

650 Route des Lucioles

F-06921 Sophia Antipolis Cedex - FRANCE

Tel.: +33 4 92 94 42 00 Fax: +33 4 93 65 47 16

Siret N° 348 623 562 00017 - APE 7112B

그라스(06) 지방청에 등록된 비영리 협회 N° w061004871

중요 고지(Important notice)

이 문서는 ETSI Search & Browse Standards 애플리케이션에서 내려받을 수 있다.

이 문서는 전자판 및/또는 인쇄판으로 제공될 수 있다. 이 문서의 전자판 및/또는 인쇄판의 내용은 ETSI의 사전 서면 승인 없이 수정해서는 안 된다. 이러한 판본 및/또는 인쇄본 사이에 내용상 차이가 존재하거나 존재한다고 인식되는 경우, 우선하는 ETSI 성과물은 ETSI deliver 저장소에 PDF 형식으로 공개된 판본이다.

사용자는 이 문서가 개정되거나 상태가 변경될 수 있음에 유의해야 한다. 이 정보는 Milestones 목록에서 확인할 수 있다.

이 문서에서 오류를 발견한 경우, Committee Support Staff 아래에 열거된 해당 서비스로 의견을 보내기 바란다.

이 문서에서 보안 취약점을 발견한 경우, 조정된 취약점 공개(CVD) 프로그램을 통해 신고하기 바란다.

면책 및 책임 제한 고지(Notice of disclaimer & limitation of liability)

이 성과물에 제공된 정보는 일반적으로 인정되는 공학 또는 기타 전문 표준과 관련 규정에 따라 그 내용을 이해하고 해석할 수 있는 적절한 수준의 경험을 갖춘 전문가만을 대상으로 한다.

제품, 서비스 또는 벤더에 관한 어떠한 권고도 하지 않으며, 그렇게 해석되어서도 안 된다.

이 성과물이 기술적으로 정확하거나 충분하다는 것, 또는 법률 및/또는 정부 규칙 및/또는 규정을 준수한다는 것에 대해 어떠한 진술이나 보증도 하지 않으며, 나아가 상품성, 특정 목적에의 적합성 또는 지식재산권 비침해에 대해서도 어떠한 진술이나 보증도 하지 않는다.

ETSI는 어떠한 경우에도 이익의 손실이나 기타 부수적 또는 결과적 손해에 대해 책임을 지지 않는다.

이 성과물에 포함된 모든 소프트웨어는 상품성, 특정 목적에의 적합성 및 지식재산권 비침해에 대한 보증을 포함하되 이에 국한되지 않는 명시적 또는 묵시적 보증 없이 "있는 그대로(AS IS)" 제공되며, ETSI는 소프트웨어의 사용 또는 사용 불능으로 인해 또는 그와 관련하여 발생하는 어떠한 손해에 대해서도 책임을 지지 않는다.

저작권 고지(Copyright Notification)

ETSI의 서면 허가에 의해 승인된 경우를 제외하고, 복사 및 마이크로필름을 포함하여 전자적 또는 기계적인 어떠한 형태나 수단으로도 이 문서의 어떠한 부분도 복제하거나 이용할 수 없다. PDF 판본의 내용은 ETSI의 서면 승인 없이 수정해서는 안 된다. 저작권 및 앞의 제한은 모든 매체에서의 복제에 적용된다.

© ETSI 2026. 모든 권리 보유.

목 차

지식재산권(IPR)

머리말

조동사 용어

1 적용 범위

2 참고 문헌

3 용어, 기호 및 약어의 정의

4 설명

4.1 개요

4.2 API 의 동작

4.3 연결 규격 및 보안 방식

4.4 보안 및 운용 환경

4.5 확장

4.6 SAE ID

4.7 API 활용사례 예

5 versions: 사용 가능한 API 버전

5.1 태그 설명

5.2 GET /kmapl/versions: 지원 API 버전 조회

6 ext_keys: 상호운용(외부 키)

6.1 태그 설명

6.2 POST /kmapl/v1/ext_keys: 외부 KMS 로 키 전달

6.3 POST /kmapl/v1/ext_keys/ack: API 요청 후 키 상태 확인 응답

6.4 POST /kmapl/v1/ext_keys/void: 외부 KMS 에 키 무효화 신호(즉, 키 폐기)

7 스키마

7.1~7.26 ack_callback_url ~ void_container

부속서 A (규범) OpenAPI 표현

이력

지식재산권(Intellectual Property Rights)

필수 특허

규범적 성과물에 필수적이거나 필수적일 가능성이 있는 지식재산권(IPR)은 ETSI에 신고되었을 수 있다. 이러한 필수 IPR에 관한 신고가 있는 경우, 해당 신고는 ETSI 회원과 비회원 모두에게 공개되며 ETSI 사무국에서 제공하는 ETSI SR 000 314 "Intellectual Property Rights (IPRs); Essential, or potentially Essential, IPRs notified to ETSI in respect of ETSI standards"에서 확인할 수 있다. 최신 갱신 사항은 ETSI IPR 온라인 데이터베이스에서 제공된다.

ETSI IPR 정책을 포함한 ETSI 지침에 따라, ETSI는 IPR 검색을 포함하여 IPR의 필수성에 관한 어떠한 조사도 수행하지 않았다. ETSI SR 000 314(또는 ETSI 웹 서버의 갱신본)에 참조되지 않은 다른 IPR로서 이 문서에 필수적이거나, 필수적일 수 있거나, 향후 필수적이 될 수 있는 IPR이 존재하지 않는다는 점은 보장할 수 없다.

상표

이 문서에는 소유자가 주장하거나 등록한 상표 및/또는 상호가 포함될 수 있다. ETSI는 ETSI의 소유로 표시된 것을 제외하고는 이들에 대한 소유권을 주장하지 않으며, 상표 및/또는 상호를 사용하거나 복제할 권리를 부여하지 않는다. 이 문서에서 해당 상표를 언급하는 것이 그 상표와 관련된 제품, 서비스 또는 조직을 ETSI가 보증함을 의미하지는 않는다.

DECT™, PLUGTESTS™, UMTS™ 및 ETSI 로고는 ETSI가 회원의 이익을 위해 등록한 상표이다. 3GPP™, LTE™ 및 5G™ 로고는 ETSI가 회원 및 3GPP 조직 파트너의 이익을 위해 등록한 상표이다. oneM2M™ 로고는 ETSI가 회원 및 oneM2M 파트너의 이익을 위해 등록한 상표이다. GSM®과 GSM 로고는 GSM 협회가 등록하고 소유한 상표이다.

머리말(Foreword)

이 그룹 규격(GS)은 ETSI 산업 규격 그룹(ISG) 양자 키 분배(QKD)가 작성하였다.

조동사 용어(Modal verbs terminology)

이 문서에서 "shall", "shall not", "should", "should not", "may", "need not", "will", "will not", "can", "cannot"은 ETSI 작성 규칙(ETSI Drafting Rules)의 3.2 절(조항 표현을 위한 동사 형태)에 기술된 대로 해석한다.

"must"와 "must not"은 직접 인용에 사용되는 경우를 제외하고 ETSI 성과물에서 사용이 허용되지 않는다.

1 적용 범위(Scope)

이 문서는 공통의 신뢰 노드에 위치한 두 키 관리 시스템이 상호운용하여 두 시스템 간에 키를 수평적으로 전달할 수 있게 하는 REST API 를 규정한다. 이 API 는 동일 벤더 시스템의 연속된 체인으로 연결되지 않은 키 관리 시스템에 공유 비밀 키를 요청하는 애플리케이션에, QKD 네트워크가 서비스를 제공할 수 있게 한다. 기반 QKD 네트워크가 노드 간에 키 자료를 어떻게 합의하는지 기술하는 것은 이 문서의 범위를 벗어난다. URI 형식, 통신 프로토콜(HTTPS), 그리고 키 자료를 포함하여 전송되는 파라미터와 응답의 JSON 데이터 형식 부호화를 기술한다. API 의 OpenAPI 기술을 이용할 수 있다.

2 참고 문헌(References)

2.1 규범적 참고 문헌

참고 문헌은 특정 판본을 지정하거나(발행일 및/또는 판 번호나 버전 번호로 식별) 지정하지 않는다. 특정 판본을 지정한 경우에는 인용된 판본만 적용된다. 특정 판본을 지정하지 않은 경우에는 해당 문서의 최신판(모든 개정 포함)이 적용된다.

예상 위치에서 공개적으로 이용할 수 없는 참조 문서는 ETSI docbox 에서 찾을 수 있다.

주 - 이 절에 포함된 하이퍼링크는 발행 시점에는 유효하였으나, ETSI 는 그 장기적 유효성을 보장할 수 없다.

다음 참조 문서는 이 문서의 적용에 필요하다.

- [1] IETF RFC 9110: STD 97 (June 2022): "HTTP Semantics".
- [2] IETF RFC 8259: STD 90 (December 2017): "The JavaScript Object Notation (JSON) Data Interchange Format".
- [3] IETF RFC 9112: STD 99 (June 2022): "HTTP/1.1".
- [4] IETF RFC 8446 (August 2018): "The Transport Layer Security (TLS) Protocol Version 1.3".
- [5] IETF RFC 3986: STD 66 (January 2005): "Uniform Resource Identifier (URI): Generic Syntax".
- [6] IETF RFC 5234: STD 68 (October 2005): "Augmented BNF for Syntax Specifications: ABNF".
- [7] IETF RFC 4648 (October 2006): "The Base16, Base32, and Base64 Data Encodings".
- [8] IETF RFC 9562 (May 2024): "Universally Unique IDentifiers (UUIDs)".
- [9] IETF RFC 9457 (July 2023): "Problem Details for HTTP APIs".
- [10] OpenAPI Initiative: "OpenAPI Specification v3.1.2 (September 2025)".

2.2 정보성 참고 문헌

참고 문헌은 특정 판본을 지정하거나(발행일 및/또는 판 번호나 버전 번호로 식별) 지정하지 않는다. 특정 판본을 지정한 경우에는 인용된 판본만 적용된다. 특정 판본을 지정하지 않은 경우에는 해당 문서의 최신판(모든 개정 포함)이 적용된다.

주 - 이 절에 포함된 하이퍼링크는 발행 시점에는 유효하였으나, ETSI 는 그 장기적 유효성을 보장할 수 없다.

다음 참조 문서는 ETSI 성과물을 구현하는 데 유용하거나 독자의 이해를 도울 수 있으나, 이 문서에 대한 적합성에는 필요하지 않다.

- [i.1] ETSI GR QKD 007: "Quantum Key Distribution (QKD); Vocabulary".
- [i.2] ETSI GS QKD 014: "Quantum Key Distribution (QKD); Protocol and data format of REST-based key delivery API".
- [i.3] ETSI GS QKD 004: "Quantum Key Distribution (QKD); Application Interface".

3 용어, 기호 및 약어의 정의

3.1 용어(Terms)

이 문서에서는 ETSI GR QKD 007 [i.1]에 제시된 용어와 다음 용어를 적용한다.

주 – 이 문서에서 정의한 용어는 ETSI GR QKD 007 [i.1]에 같은 용어의 정의가 있는 경우 그보다 우선한다.

연동 노드(interworking node): 둘 이상의 QKD 네트워크에 속하는 노드로, 노드 내에서 연동하는 각 QKD 네트워크의 키 관리 개체를 포함하는 것

키 관리 개체(key management entity): 하나 이상의 다른 키 관리 개체와 협력하여 네트워크에서 키를 관리하는 개체

보안 애플리케이션 개체(secure application entity): 하나 이상의 다른 보안 애플리케이션 개체와 협력하여 동작하는 하나 이상의 애플리케이션을 위해, 키 관리 개체에 하나 이상의 키를 요청하는 개체

3.2 기호(Symbols)

공란(Void).

3.3 약어(Abbreviations)

이 문서에서는 다음 약어를 적용한다.

API	Application Programming Interface
HEXDIG	HEXadecimal DIGits
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IANA	Internet Assigned Numbers Authority
IETF	Internet Engineering Task Force
IP	Internet Protocol
JSON	JavaScript Object Notation
KME	Key Management Entity
KMS	Key Management System
PEN	Private Enterprise Number
QKD	Quantum Key Distribution
REST	REpresentational State Transfer
SAE	Secure Application Entity
TLS	Transport Layer Security
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
UTF-8	UCS Transformation Format 8 bits
UUID	Universally Unique IDentifier

4 설명(Description)

4.1 개요

이 문서는 양자 키 분배(QKD) 네트워크가 보호된 운용 환경 내에 함께 위치할 때 키를 교환하기 위해 사용하는 키 관리 개체(KME)용 통신 프로토콜과 데이터 형식을 규정한다. 활용사례는 다음과 같다.

- KME 가 서로 다른 QKD 네트워크에 속하며 연동 노드 내에 위치하는 경우.
- KME 가 하나의 QKD 네트워크 내에서 서로 다른 정책 도메인에 속하는 경우.
- KME 가 그 밖의 이유로 호환되지 않는 경우(예: 서로 다른 벤더).

활용사례에는 서로 다른 네트워크 운영자와 정책 도메인 간의 연동이 포함되므로, KME 가 교환해야 하는 정보는 의도적으로 제한된다.

단순하고 확장 가능하며 널리 배포되어 많은 개발자 커뮤니티에 익숙한 방식으로서 REST(REpresentational State Transfer) API 를 규정한다. 이 REST 기반 API 는 URI 의 형식, 통신 프로토콜 "HTTPS"(즉, TLS 를 적용한 HTTP), 그리고 키 자료를 포함하여 요청 및 응답 내용의 데이터 형식과 부호화를 규정한다.

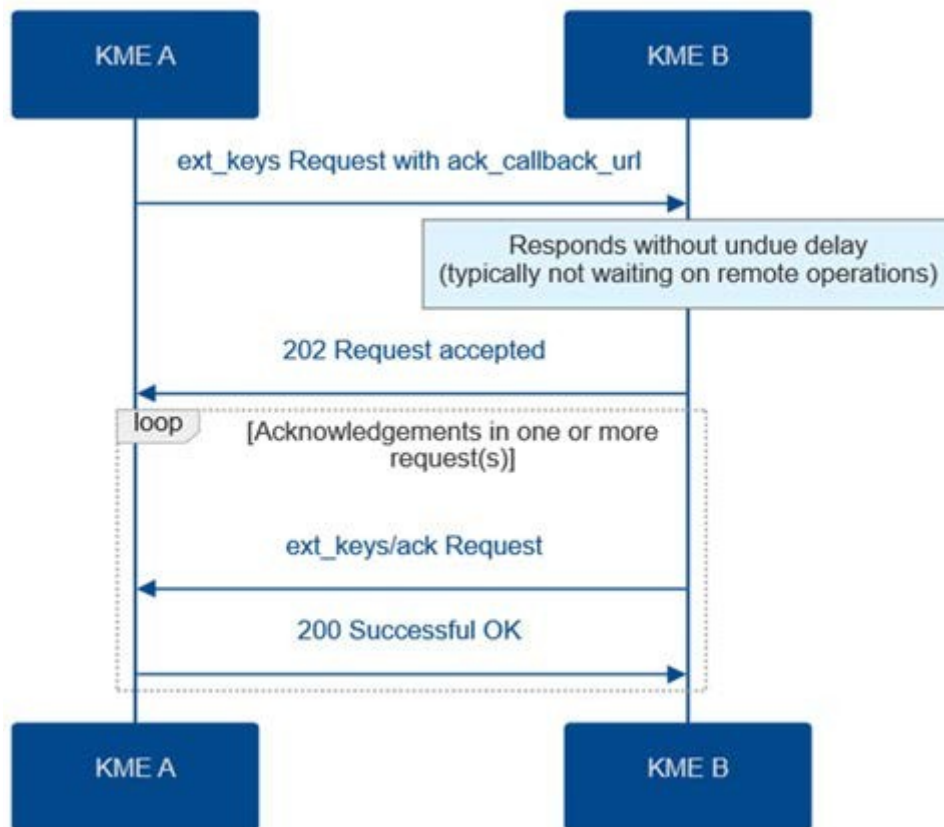
4.2 API 의 동작

QKD 네트워크의 주된 목적은 보안 애플리케이션 개체(SAE)라 불리는 인가된 암호 애플리케이션에 공유된 안전한 무작위 키를 전달하는 것이다. KME 에서 SAE 로 키를 전달하기 위한 API 는 ETSI GS QKD 014 [i.2]와 ETSI GS QKD 004 [i.3]에 규정되어 있다.

SAE 가 키 공유를 요청하는 KME 들 사이에 직접 QKD 링크가 없는 경우, QKD 네트워크 내 KME 들은 보통 키를 릴레이한다. 이를 안전하게 수행하기 위해 QKD 키를 소비할 수 있다. 서로 다른 KME 집합은 서로 완전히 호환되지 않을 수 있다. KME 집합은 별도로 관리될 수도 있다(예: 내부 네트워크 정보의 불필요한 공유를 제한하려는 서로 다른 네트워크 운영자에 의해).

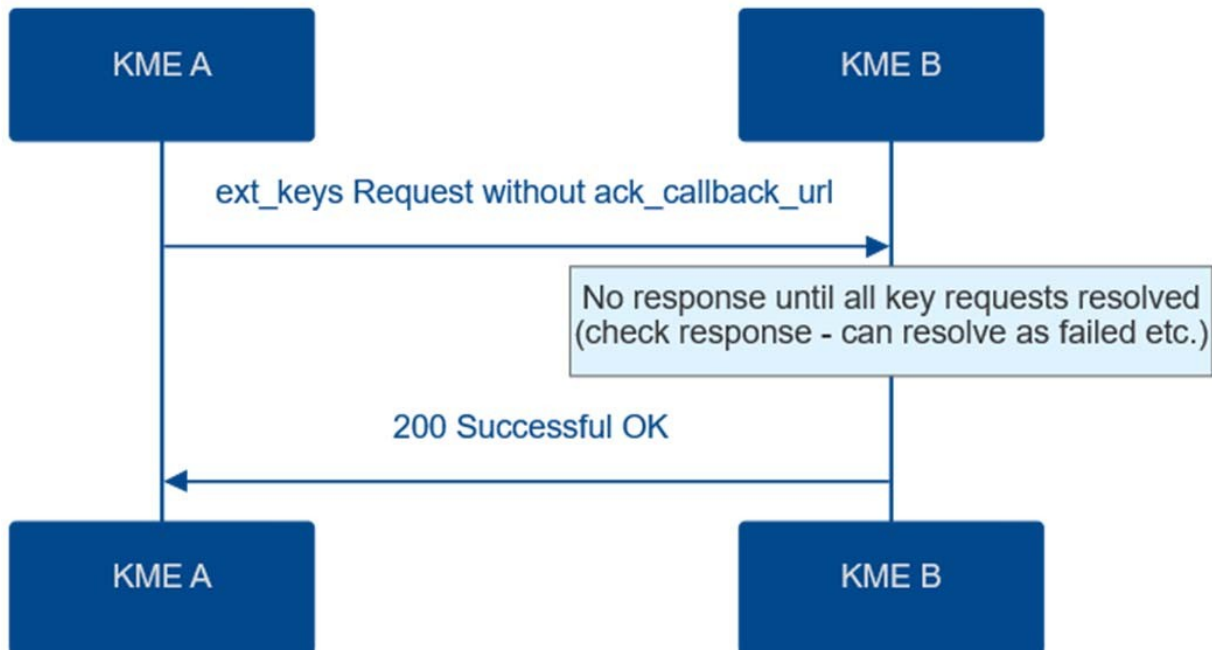
이러한 경우, 비호환성이나 정책 등의 이유로 달리 상호운용할 수 없는 서로 다른 KME 집합 간에 KME 가 키를 수평적으로 전달할 수 있도록 표준화된 인터페이스가 필요하다. 이 문서에 기술된 API 는 보안 운용 환경 내의 KME 간 키 전달을 가능하게 한다. 이를 통해 서로 다른 QKD 네트워크, 정책 도메인 등의 KME 에 연결된 두 개(또는 그 이상)의 SAE 에 공유 키를 전달할 수 있다.

이 문서에 기술된 API 메서드에는 비동기 모드와 동기 모드의 두 가지 주요 동작 모드가 있다. 비동기 모드에서는 요청이 논블로킹 방식으로 처리되어, KME 가 부당한 지연 없이 요청을 수락하고 "202"('Request accepted') 응답을 발행하여 요청 연결을 닫을 수 있다. 실제 키 처리는 이후 백그라운드에서 수행되며, KME 는 최초 요청을 보낸 KME 에 하나 이상의 요청을 다시 보내 결과의 세부 사항을 제공함으로써 해당 키 처리 결과를 확인 응답한다. 이 모드는 트래픽이 많은 환경이나 요청 처리에 시간이 오래 걸릴 수 있는 경우에 특히 유리하다. 요청자를 블로킹하지 않으며 KME 간에 동시에 열려 있는 연결 수를 최소화한다. 그 결과 이 모드는 확장성과 함께 비정상 상황에서의 예측 가능한 성능을 달성하는 데 도움이 될 수 있다. 비동기 모드의 개요 메시지 시퀀스 다이어그램은 (그림 4.2-1)과 같다.



(그림 4.2-1) 필수 비동기 모드의 성공 시 개요 메시지 시퀀스 다이어그램

반대로 동기 모드에서는 모든 결과가 단일 응답으로 반환되므로, 요청자는 요청의 모든 부분이 완료되거나 실패할 때까지 기다려야 한다. 이 모드는 KME 간에 더 직접적이고 간단한 상호작용을 제공하며 결과를 거의 즉시 반환할 수 있는 경우에 적합할 수 있으나, 대규모이거나 혼잡한 네트워크에서는 확장성이 제한될 수 있다. 선택 사항인 동기 모드의 메시지 시퀀스 다이어그램은 (그림 4.2-2)와 같다.



(그림 4.2-2) 선택 동기 모드의 성공 시 개요 메시지 시퀀스 다이어그램

상호운용성을 보장하기 위해, 모든 KME 는 이 문서에 정의된 API 메서드에 대해 비동기 동작 모드를 지원해야 한다.

KME 는 특정 구현(들)을 위해 필수인 비동기 모드에 더하여 동기 동작 모드를 선택 모드로 지원할 수 있다.

KME 는 자 신 이 받은 "ext_keys" 호 출 로 부 터 발 생 하 였 고 "400"('Bad request format') 또는 "401"('Unauthorised') 응답을 반환한 요청 내의 어떤 키에 대해서도 연관을 유지해서는 안 된다.

KME 는 "400"('Bad request format') 또는 "401"('Unauthorised') 응답이 반환되지 않는 한, "ext_keys" 요청의 결과로 키가 "initiator_sae_id" 및 "target_sae_id"와 연관되었을 수 있다고 가정해야 한다. 이 두 응답 중 하나를 성공적으로 받지 않는 한, KME 는 요청 내 키의 할당이 잠재적으로 생성되었을 수 있다고 가정해야 한다.

주 1 – 잠재적 연관은 KME 내에 존재할 수도 있고, KME 와 상호작용한 다른 개체가 생성했을 수도 있다. 타임아웃이 발생한 경우에도 잠재적 연관이 생성되었을 수 있다.

KME 는 생성되었을 수 있는 잠재적 할당을 고려하지 않고 "ext_keys" 호출에 포함되었던 키를 재할당하지 않는 것이 좋다.

주 2 – KME 는 보통 키가 의도된 모든 "target_sae_id"에 성공적으로 릴레이되었음을 알기 전까지는 SAE 에 키를 내주지 않는다.

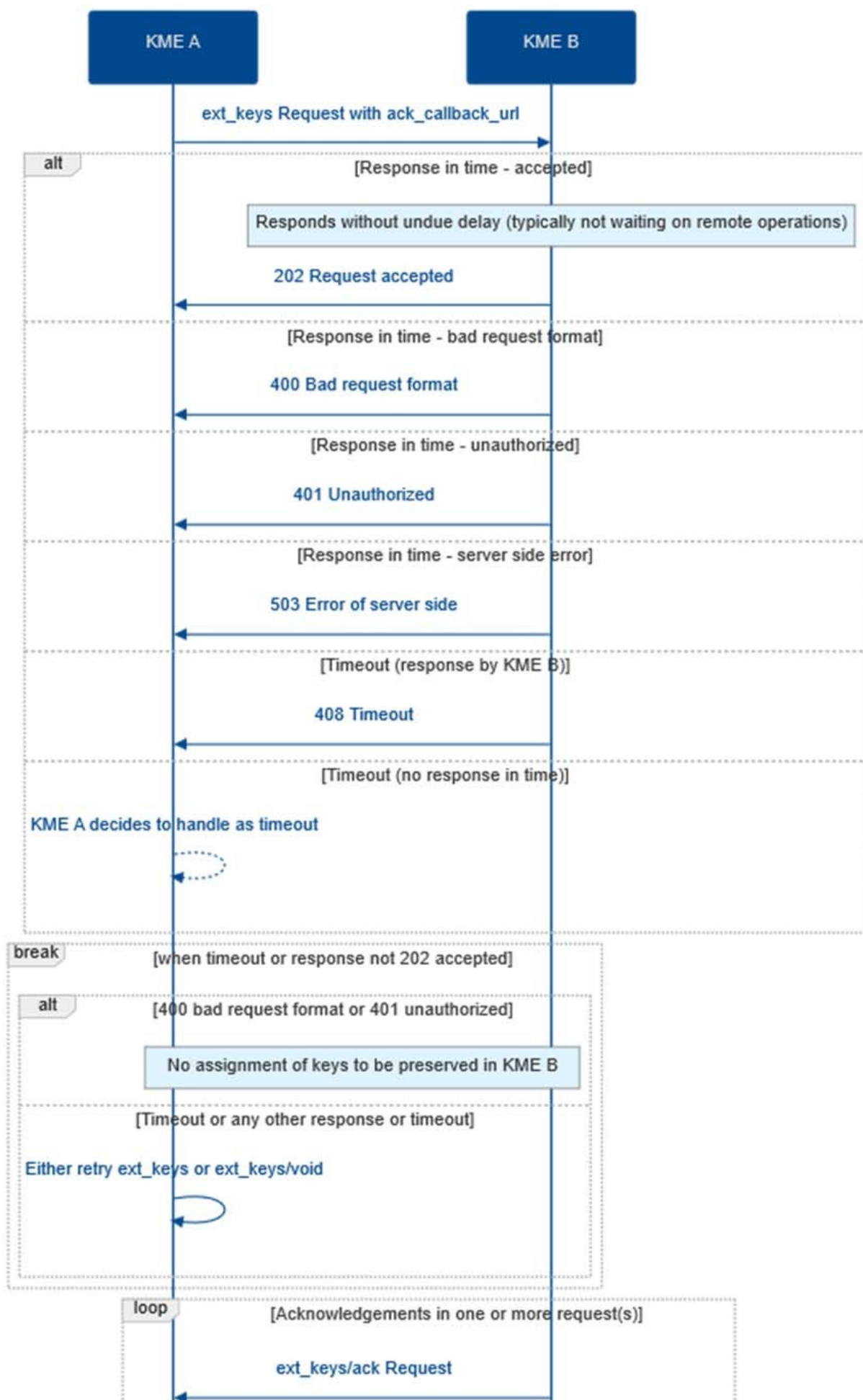
"ext_keys" 요청에서 "400"('Bad request format') 또는 "401"('Unauthorised') 이외의 오류나 타임아웃이 발생한 경우, KME 는 영향을 받은 모든 키에 대해 이후의 "ext_keys" 요청으로 실패한 요청을 재시도하거나, 자신의 네트워크(들) 내에서 영향을 받은 키를 무효화하고 "ext_keys/void" 요청을 발행해야 한다.

"400"('Bad request format') 또는 "401"('Unauthorised') 응답의 경우, KME 는 선택적으로 실패한 요청을 재시도할 수 있다.

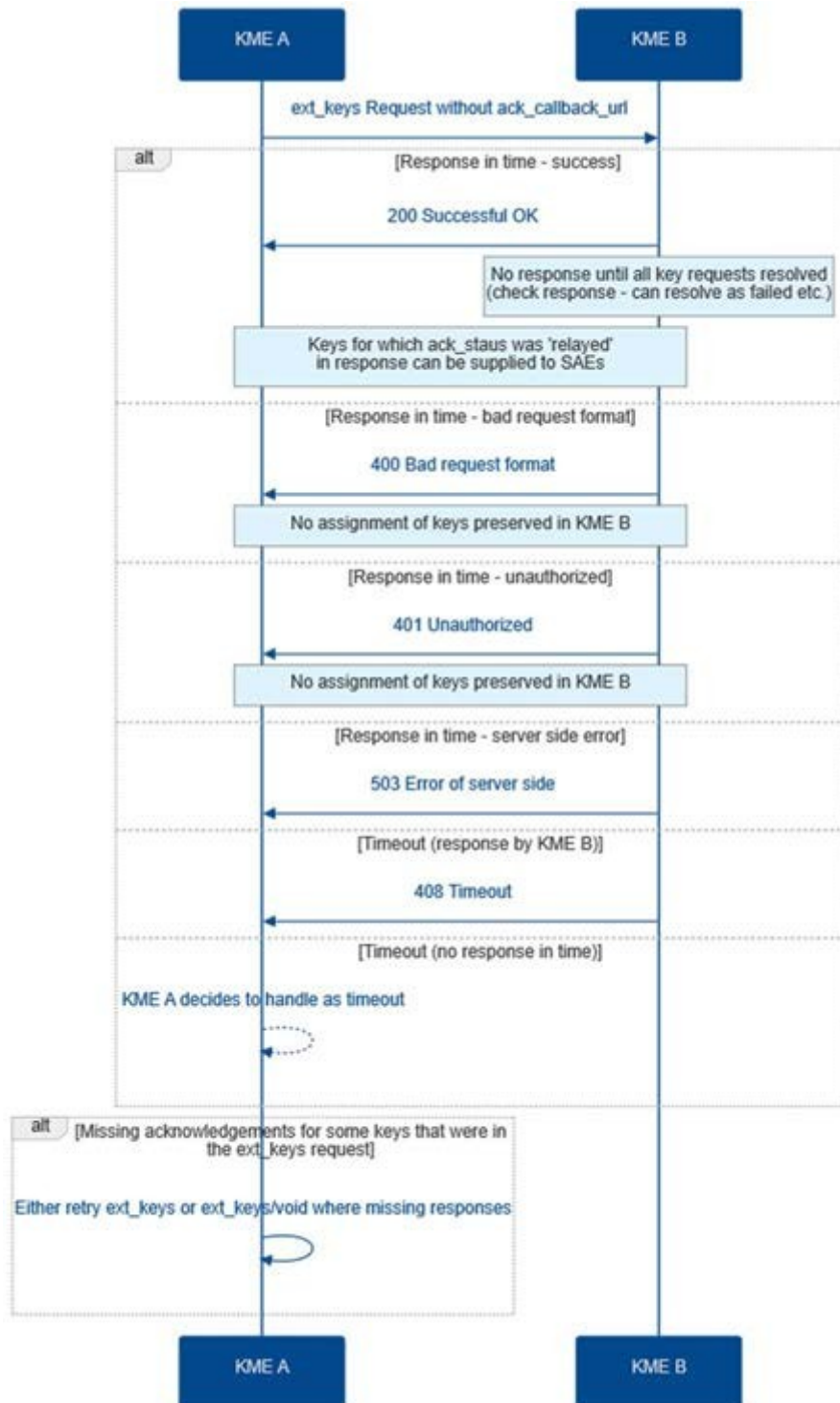
"ext_keys/ack" 요청에서 오류나 타임아웃이 발생한 경우, KME 는 영향을 받은 모든 키에 대해 이후의 "ext_keys/ack" 요청으로 실패한 요청을 재시도하거나, 자신의 네트워크(들) 내에서 영향을 받은 키를 무효화하고 "ext_keys/void" 요청을 발행해야 한다.

주 3 – "ext_keys/ack"의 경우, "ext_keys/void" 호출은 관련 "ext_keys" 요청을 발행한 KME 를 향해 반대 방향으로 발행된다.

보다 상세한 시퀀스 다이어그램은 필수 비동기 모드에 대해 (그림 4.2-3), 선택 동기 모드에 대해 (그림 4.2-4)에 제시한다.



(그림 4.2-3) 필수 비동기 모드의 메시지 시퀀스 다이어그램



(그림 4.2-4) 선택 동기 모드의 메시지 시퀀스 다이어그램

4.3 연결 규격 및 보안 방식

API의 공통 연결 규격은 <표 4.3-1>과 같다.

<표 4.3-1> 공통 연결 규격

Name	설명
Communication Protocol	"HTTPS": HTTP/1.1 과, 선택적으로 이에 더해 상위 버전; TLS v1.3 이상
Content-Type	application/json; charset=UTF-8

이 API에 규정된 모든 통신 기능은 IETF RFC 9110 [1]과 IETF RFC 9112 [3]에 규정된 HTTP/1.1 프로토콜에 전송 계층 보안(TLS)을 적용하여 구현해야 한다.

주 1 – 이 조합을 흔히 HTTPS 프로토콜이라 한다.

HTTP/1.1에 더하여 HTTP 프로토콜의 상위 버전을 구현할 수 있다.

IETF RFC 8446 [4]에 규정된 TLS v1.3 이상을 사용해야 한다.

연결 수립 시 KME 간에 인증서를 사용한 상호 인증을 수행해야 한다. 두 KME는 각각 상대 KME 인증서의 유효성을 검증하고 인증서로부터 그 신원을 확인해야 한다.

요청과 응답에 본문 내용이 포함되는 경우, 이는 "application/json"으로 부호화해야 한다.

주 2 – OpenAPI는 버전 3.1에서 상호 TLS 보안 규정 지원을 도입하였다. 이 OpenAPI 기술의 다른 부분은 이전 도구에서 사용하기 위해 OpenAPI 버전 3.0으로 쉽게 변환할 수 있으나, 어떤 구현에서든 Security Scheme Object에 규정된 보안은 이 4.3 절에 따라 수동으로 재정의되도록 보장해야 한다.

4.4 보안 및 운용 환경

이 API는 동일한 사이트(두 QKD 네트워크 간에 사용될 때는 연동 노드) 내에 위치한 KME 간에 사용하기 위한 것이다. KME와, 이 문서가 규정하는 KME 간 통신은 보안 운용 환경 내에 유지되는 것이 좋다. 운용 환경에 대해 적절한 보안 조치를 고려하는 것이 좋다.

키 처리 및 관리에 대한 일반적인 보안 요구사항은 이 문서의 범위를 벗어난다.

4.5 확장(Extensions)

일부 메서드는 확장을 제공할 수 있게 한다. 확장은 벤더가 규정할 수 있으며, KME는 구현했다고 주장하는 모든 확장의 규격을 따라야 한다. 관련되는 경우, 여기에는 확장의 규격에서 변경 불가로 표시된 정보를 변경 없이 전달하는 것이 포함된다.

속성은 어떠한 유형이든 될 수 있으나, 그 이름은 고유해야 하며 벤더 접두사로 시작해야 한다. 벤더 접두사는 대문자 "E" 다음에 IANA 사설 기업 번호(PEN)와 밑줄이 오는 형태여야 한다. PEN을 할당받은 조직은 확장을 자체 목적으로 사용할 수도 있고, 다른 조직도 구현할 수 있도록 확장의 규격을 다른 조직에 제공할 수도 있다. 어떤 조직도 다른 조직의 PEN을 포함하는 벤더 접두사로 시작하는 확장을, 그 PEN과 연관된 조직의 규격을 따르지 않고 사용해서는 안 된다.

ETSI는 향후 확장을 규정할 수 있으나, 그렇게 하기 전까지 ETSI의 IANA 사설 기업 번호 "13019"는 예약되어 있으며 사용해서는 안 된다.

주 – 확장은 키 릴레이를 촉발한 SAE 요청에서 비롯될 수도 있고, KME가 추가할 수도 있다(예: 메타데이터 추가 등). 확장의 규격에 있는 요구사항은 이후 호출에서 KME가 확장을 추가하는 것을 금지할 수 있다. 확장은 KME 내에서 사전(dictionary)으로 역직렬화될 수 있다. 속성 이름과 그 값 모두 의미를 가질 수 있다.

관련 스키마에는 "extension", "extension_mandatory", "extension_optional"이 있다.

4.6 SAE ID

SAE ID는 연결된 모든 QKD 네트워크 집합 전체에서 고유해야 한다. 경우에 따라 SAE ID가 중복되면 의도된 SAE가 아닌 다른 SAE가 키를 얻게 될 수 있다. 달리 안전하게 구현된 KME로부터 ETSI GS QKD 014 [i.2]의 API를 사용하여 SAE ID 중복 상황에서 다른 SAE를 위한 키를 얻으려면, 악의적인 대상 SAE는 적절한 시점의 요청에서 다른 대상 SAE와 연관된 유효한 "key_id"를 지정해야 한다. 그러나 일부 상황에서 잠재적 완화책(예: "key_id"를 기밀로 유지)이 적용되더라도, SAE ID 중복은 신뢰성 있는 전달을 저해할 수 있다.

연결된 QKD 네트워크가 SAE ID의 고유성을 어떻게 보장하는지는 이 문서의 범위를 벗어난다.

SAE ID("initiator_sae_id"와 "target_sae_id")는 길이가 64자를 초과해서는 안 된다. SAE가 다른 프로토콜의 길이 제한 파라미터로 SAE ID를 전달하려 할 가능성이 있는 경우, 연결된 QKD 네트워크는 SAE ID 길이를 추가로 제한하는 것을 고려할 수 있다.

SAE ID 는 IETF RFC 5234 [6]를 참조하여 IETF RFC 3986 [5]에 규정된 URI 내에서 허용되지 않는 문자를 포함해서는 안 된다. SAE ID 가 URI 일 필요는 없다.

주 - IETF RFC 3986 [5]은 비예약 문자 "unreserved = (ALPHA / DIGIT / "-" / "." / "_" / "~")", 예약 일반 구분 문자 "gen-delims = (":" / "/" / "?" / "#" / "[" / "]" / "@")", 하위 구분 문자 "sub-delims = ("!" / "\$" / "&" / "'" / "(" / ")" / "*" / "+" / "," / ";" / "=")", 그리고 퍼센트 부호화 문자 "pct-encoded = "%" HEXDIG HEXDIG"를 허용한다. "ALPHA = (%x41-5A / %x61-7A; A-Z / a-z)", "DIGIT = %x30-39; 0-9", "HEXDIG = (DIGIT / "A" / "B" / "C" / "D" / "E" / "F")"의 정의는 IETF RFC 5234 [6]를 참조한다.

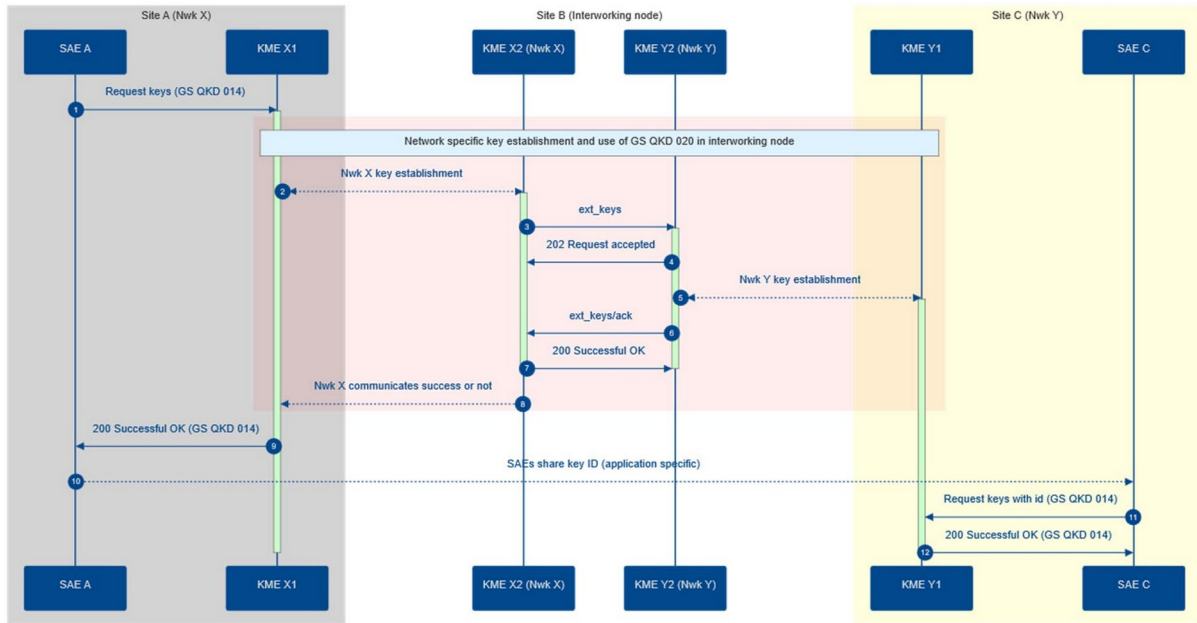
4.7 API 활용사례 예

(그림 4.7-1)은 세 노드 네트워크의 연동 노드에서 KME 간에 키를 전달하기 위해 "ext_keys" 연산을 사용하는 활용사례를 보여준다. 이 네트워크는 둘 다 연동 노드에 존재하는 두 네트워크 X와 Y로 구성된다. 이 활용사례에서는 ETSI GS QKD 014 [i.2]를 사용한 애플리케이션으로의 키 전달을 고려한다.

두 QKD 네트워크 간의 비동기 키 전달은 어떤 SAE 프로토콜이나 작업 흐름과도 분리되어 있다. 이 활용사례는 개시 SAE 가 키를 요청하는 반응형 패턴을 고려한다. KME 노드가 개시 SAE 로 동작하여, 이후 클라이언트 SAE 에 키를 전달하는 데 사용할 수 있는 하나 이상의 공유 키 버퍼를 채우기 위해 키를 요청하는 능동형 패턴의 활용사례도 가능하다

- 1) 사이트 A 내에서 SAE A 가 QKD 네트워크 X 의 KME X1 에, 다른 QKD 네트워크 Y 가 서비스하는 별도의 사이트 C 에 있는 SAE C 와 공유할 키를 로컬로 요청한다.
- 2) QKD 네트워크 X 는 QKD 키를 사용하는 연산을 수행하여, SAE A 의 사이트에 있는 KME X1 과 사이트 B 의 연동 노드에 있는 KME X2 사이에 공유 키를 수립한다. 통신은 QKD 네트워크 X 내 시스템의 독자 방식일 수 있으며, 세부 사항은 (그림 4.7-1)에 표시하지 않았다.
- 3) 사이트 B 내에서 KME X2 는 이 문서에 규정된 API 를 사용하여 KME Y2 의 "ext_keys"에 요청을 발행한다.
- 4) KME Y2 는 즉시 HTTP 상태 코드 "202"('Request accepted')로 응답한다.
- 5) QKD 네트워크 Y 는 QKD 키를 사용하는 연산을 수행하여 사이트 C 의 KME Y1 로 키를 전달한다. 통신은 QKD 네트워크 Y 내 시스템의 독자 방식일 수 있으므로, 세부 사항은 (그림 4.7-1)에 표시하지 않았다.
- 6) QKD 네트워크 Y 가 KME Y1 에서 키를 사용할 수 있음을 확인하면, KME Y2 는 요청이 성공했음을 확인 응답하기 위해 KME X2 의 "ext_keys/ack"에 요청을 발행한다.
- 7) KME X2 는 확인 응답을 받았음을 확인하기 위해 즉시 HTTP 상태 코드 "200"('Successful OK')으로 응답한다.
- 8) QKD 네트워크 X 는 성공 결과를 KME X1 에 다시 전달한다. 통신은 QKD 네트워크 X 내 시스템의 독자 방식일 수 있으며, 세부 사항은 (그림 4.7-1)에 표시하지 않았다.
- 9) KME X1 은 SAE A 의 최초 요청에 응답하여 SAE A 에 키를 반환한다.
- 10) SAE A 는 키 ID 를 SAE C 에 전달한다. 통신은 애플리케이션 시스템의 독자 방식일 수 있으므로, 세부 사항은 (그림 4.7-1)에 표시하지 않았다.
- 11) SAE C 는 키 ID 를 사용하여 사이트 C 내에서 로컬로 KME Y1 에 키를 요청한다.
- 12) KME Y1 은 SAE C 에 키로 응답한다. 이제 SAE A 와 SAE C 모두 동일한 키를 공유하며, 이를 암호 용도로 소비할 수 있다.

이러한 방식으로 연동 노드 내에서 "ext_keys"를 사용하여 SAE A 와 SAE C 사이에 키가 공유된다.



(그림 4.7-1) 연동 노드를 경유하여 수립되는 실시간 ETSI GS QKD 014 [i.2] 키 요청 활용사례의 시퀀스 다이어그램

5 versions: 사용 가능한 API 버전

5.1 태그 설명

KME 에서 사용할 수 있는 이 API 의 버전을 조회하는 데 사용할 수 있는 버전 없는 경로.

5.2 GET /kmapl/versions: 지원 API 버전 조회

5.2.1 설명

KME 가 지원하는 API 버전의 목록을 반환한다.

KME 가 다른 KME 에 요청할 때는 두 KME 가 모두 지원하는 가장 최근 버전의 API 를 사용하는 것이 좋다.

SAE 는 이 엔드포인트를 사용하여 KME 가 지원하는 API 버전을 확인할 수 있다.

역자 주 - 원문은 이 문장의 주체를 "SAE"로 적고 있어 그대로 옮겼으나, 이 API 는 KME 간 인터페이스이므로 KME 를 의미하는 것으로 보인다.

5.2.2 응답(Responses)

5.2.2.1 200 성공(Successful OK) 응답 application/json

응답 본문은 <표 5.2.2.1-1>에 나타난 "version_container" 스키마와 일치해야 하며, IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

<표 5.2.2.1-1> GET /kmapl/versions 에 대한 200 응답의 데이터 유형 "version_container"(application/json)

Field Name	Type	설명
versions	array of string schema: versions	REQUIRED 버전 "versions"는 지원하는 API 버전 "version"의 배열이어야 한다. Maximum items: 1 000
capabilities	array of string schema: capabilities	능력 "capabilities"는 각각 호스트가 지원하는 "capability"를 나타내는 문자열의 배열이어야 한다. 호스트가 선택 사항인 동기 모드를 지원하는 경우 이 능력을 "synchronous_mode"로 보고하는 것이 좋다. 클라이언트는 동기 모드 요청을

Field Name	Type	설명
		보내기 전에 이 능력을 확인하는 것이 좋다. Maximum items: 1 000
extension	object schema: extension	확장 "extension"은 각각 ETSI 또는 다른 조직(예: 벤더)이 규정한 확장을 나타내는 임의 개수의 속성을 담은 객체여야 한다. KME 는 일련의 확장을 구현하도록 선택할 수 있다. 확장의 규격에는 요구사항이 포함될 수 있으며, 확장을 구현하는 KME 는 그러한 요구사항을 준수해야 한다. 추가 요구사항은 4.5 절에 제시되어 있다. Minimum properties: 1 Maximum properties: 1 024

예: "/kmapl/versions"에 대한 "200" 상태 코드 "version_container" 응답 본문의 예(미디어 유형 "application/json"):

```
{
  "versions": [
    "v1"
  ],
  "capabilities": [
    "synchronous_mode"
  ],
  "extension": {
    "E32473_extension1": "Some string",
    "E32473_extension2": {
      "property1": 10111,
      "property2": "Some text",
      "property3": {
        "subprop1": 21,
        "subprop2": true
      }
    }
  }
}
```

5.2.2.2 401 인가 실패(Unauthorized) 응답 application/json

응답 본문은 <표 5.2.2.2-1>에 나타난 "problem_details" 스키마와 일치해야 하며, IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

<표 5.2.2.2-1> GET /kmapl/versions 에 대한 401 응답의 데이터 유형 "problem_details"(application/json)

Field Name	Type	설명
type	string	REQUIRED 문제 유형을 식별하는 URI 참조. Format: uri Maximum length: 1 024
status	integer	이 문제 발생에 대해 원 서버가 생성한 HTTP 상태 코드. Format: int32 Minimum: 100 Maximum: 599
title	string	사람이 읽을 수 있는 문제 유형의 짧은 요약. Maximum length: 1 024
details	object	문제에 관한 세부 정보를 전달하기 위한 하나 또는 여러 개의 문자열 속성을 담은 객체. 일반적으로 각 문자열 속성의 이름은 주제를 식별하고, 문자열 값은 더 자세한 정보를 제공한다. "details"는 문자열 사전으로 역직렬화할 수 있다. Minimum properties: 1 Maximum properties: 1 024

예: "/kmapl/versions"에 대한 "401" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```
{
  "type": "https://qkd.etsi.org/gs020-interop-kms/unauthorized",
  "status": 401,
  "title": "unauthorized",
  "details": {
    "unauthorized": "User-supplied certificate is invalid."
  }
}
```

5.2.2.3 503 서버 측 오류(Error of server side) 응답 application/json

응답 본문은 <표 5.2.2.3-1>에 나타난 "problem_details" 스키마와 일치해야 하며, IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

<표 5.2.2.3-1> GET /kmapl/versions 에 대한 503 응답의 데이터 유형 "problem_details"(application/json)

Field Name	Type	설명
type	string	REQUIRED 문제 유형을 식별하는 URI 참조. Format: uri Maximum length: 1 024
status	integer	이 문제 발생에 대해 원 서버가 생성한 HTTP 상태 코드. Format: int32 Minimum: 100 Maximum: 599
title	string	사람이 읽을 수 있는 문제 유형의 짧은 요약. Maximum length: 1 024
details	object	문제에 관한 세부 정보를 전달하기 위한 하나 또는 여러 개의 문자열 속성을 담은 객체. 일반적으로 각 문자열 속성의 이름은 주제를 식별하고, 문자열 값은 더 자세한 정보를 제공한다. "details"는 문자열 사전으로 역직렬화할 수 있다. Minimum properties: 1 Maximum properties: 1 024

예 1: "/kmapl/versions"에 대한 "503" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```
{
  "type": "https://qkd.etsi.org/gs020-interop-kms/server-side-general-error",
  "status": 503,
  "title": "server side general error",
  "details": {
    "server_side_general_error": "The server encountered a general failure and cannot respond."
  }
}
```

예 2: "/kmapl/versions"에 대한 "503" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```
{
  "type": "https://qkd.etsi.org/gs020-interop-kms/key-routing-error",
  "status": 503,
  "title": "key routing error",
  "details": {
    "no_route_multiple_targets": "Routes not available to support these multiple target SAEs."
  }
}
```

6 ext_keys: 상호운용(외부 키)

6.1 태그 설명

다른 KME 에 키를 전달하거나, 키 상태를 확인 응답하거나, 키를 무효화하기 위한 버전 있는 경로의 모음.

6.2 POST /kmap/v1/ext_keys: 외부 KMS 로 키 전달

6.2.1 설명

키 자료와 관련 데이터로 구성된 확장 키 요청 컨테이너 "ext_key_container"를 다른 KME 에 전달하여, 지정된 대상 SAE 에 (필요한 경우 릴레이를 통해) 키가 전달되도록 한다. 확장 키 요청 컨테이너에는 개시 SAE 에 전달될 키와 일치하는 키가 담긴다.

유효한 요청을 받으면, 필수 비동기 모드의 경우 KME 는 부당한 지연 없이 "202"('Accepted')로 응답하는 것을 목표로 해야 하며, 이후 키가 실제로 전달되면(또는 전달에 실패하면) 지정된 "ack_callback_url" 엔드포인트로 별도의 호출(또는 여러 호출)을 발행해야 한다.

비동기 모드에 더하여 동기 모드를 구현한 경우, KME 는 동기 모드 요청(이러한 요청에서는 "ack_callback_url"이 생략되었을 것이다)의 키에 대한 결과를 확인 응답하기 위해 "ext_keys/ack"를 사용해서는 안 된다. 대신 KME 는 "200"('Successful OK')으로 응답해야 한다.

추가 조사 없이도 요청이 유효하지 않음을 아는 경우(예: 컨테이너 형식이 유효하지 않거나, KME 가 유효한 경로를 알지 못하는 개시/대상 SAE ID 가 포함된 경우) "400" 오류를 반환할 수 있다.

6.2.2 요청 본문(Request Body): application/json

요청 본문은 <표 6.2.2-1>에 나타난 "ext_key_container" 스키마와 일치해야 하며, IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

<표 6.2.2-1> POST /kmap/v1/ext_keys 요청의 데이터 유형 "ext_key_container"(application/json)

Field Name	Type	설명
keys	array of object schema: keys	REQUIRED 키 "keys"는 "key"의 배열이어야 한다.
initiator_sae_id	string schema: initiator_sae_id	REQUIRED 개시 SAE ID "initiator_sae_id"는 하나 이상의 대상 SAE 와 키 공유를 최초로 요청한 SAE 의 ID 여야 한다. "ext_keys"에서 키를 참조하는 데 사용할 수 있다. 또한 "ext_keys/void"나 "ext_keys/ack"에서 이전 "ext_keys" 호출(들)의 키를 참조하는 데에도 사용할 수 있다. KME 는 (내부) SAE 로 동작하여, 역시 SAE 로 동작하는 다른 KME 와 키 공유를 요청할 수 있다. 요청의 출처에 개시 SAE 와 대상 SAE 의 구별이 적용되지 않는 경우, 구현은 임의로 하나를 "initiator_sae_id"로, 나머지를 "target_sae_id"로 지정할 수 있다. SAE ID 는 본문 4.6 절의 요구사항을 따라야 한다. Maximum length: 64
target_sae_ids	array of string schema: target_sae_ids	REQUIRED "target_sae_ids"는 요청과 관련된 대상 SAE 의 ID 인 "target_sae_id"의 배열이어야 한다. 대상은 하나 또는 여러 개를 지정할 수 있다. 개시 SAE 와 더불어 각 대상 SAE 는 키의 동일한 사본을 받을 자격이 있는 SAE 이다. KME 는 대상 SAE 를 하나만 지원(배열에 단일 SAE ID)하거나 지원하는 대상 SAE 의 최대 수를 제한할 수 있다. 가용 KME 의 능력을 초과하는 요청은 "503"('Error of server side') 응답 코드로 거부될 수 있다.
ack_callback_url	string schema: ack_callback_url	확인 응답 콜백 URL "ack_callback_url"은 요청의 전부 또는 일부가 완료되거나 실패한 후 확인 응답(들)을 보낼 URL 이다. 필수인 비동기 모드에서는 이 파라미터가 필수이며 지정되어야 한다.

Field Name	Type	설명
		이 파라미터가 생략되면 요청은 선택 사항인 동기 모드로 엔드포인트를 호출하는 요청으로 해석되어야 한다. 그렇지 않으면 필수인 비동기 모드로 엔드포인트를 호출하는 요청으로 해석되어야 한다. 필수 비동기 모드와 선택 동기 모드의 동작은 4.2 절을 참조한다. 새로운 KME 에 "ack_callback_url" 없이 동기 모드 요청을 보내기 전에, "/kmapi/versions" 엔드포인트를 호출하여 새 KME 가 선택 사항인 동기 모드 "capability"를 지원하는지 확인하는 것이 좋다. "ack_callback_url"을 제공하는 경우, 프로토콜로 시작하는, 즉 "https://"로 시작하는 완전한 URL 이어야 한다. authority 에는 도메인이나 IP 주소 외에 선택적으로 포트 번호를 포함할 수 있다. Format: uri Maximum length: 1 024
extension_mandatory	object schema: extension_mandatory	확장 "extension_mandatory"는 각각 ETSI 또는 다른 조직(예: 벤더)이 규정한 확장을 나타내는 임의 개수의 속성을 담은 객체여야 하며, KME 는 이를 처리하거나, "unsupported_mandatory_extension"이라는 "details" 항목을 추가하여 "503"("Error of server side response")으로 요청을 거부해야 한다. 수신 KME 가 "extension_mandatory" 확장의 규격에 있는 모든 요구사항을 인식하고 구현하지 않는 한, 요청을 거부해야 한다. KME 는 요청에 대해 구현했다고 주장하는 모든 확장의 규격을 따라야 한다. 관련되는 경우, 여기에는 확장의 규격에서 변경 불가(immutable)로 표시된 정보를 변경 없이 전달하는 것이 포함된다. 추가 요구사항은 4.5 절에 제시되어 있다. Minimum properties: 1 Maximum properties: 1 024
extension_optional	object schema: extension_optional	확장 "extension_optional"은 각각 ETSI 또는 다른 조직(예: 벤더)이 규정한 확장을 나타내는 임의 개수의 속성을 담은 객체여야 하며, KME 는 해당 확장을 지원하지 않더라도 이를 전달할 수 있다. 키 관리 개체가 "extension_optional" 확장의 규격에 있는 모든 요구사항을 인식하고 구현하지 않는 경우, 그 "extension_optional" 확장을 변경 없이 전달해야 한다. KME 는 요청에 대해 구현했다고 주장하는 모든 확장의 규격을 따라야 한다. 관련되는 경우, 여기에는 확장의 규격에서 변경 불가(immutable)로 표시된 정보를 변경 없이 전달하는 것이 포함된다. 추가 요구사항은 4.5 절에 제시되어 있다. Minimum properties: 1 Maximum properties: 1 024

예: "/kmapi/v1/ext_keys"에 대한 "ext_key_container" 요청 본문의 예(미디어 유형 "application/json"):

```
{
  "keys": [
    {
      "key_id": "550e8400-e29b-41d4-a716-446655440000",
      "value": "wHHVxRwDJs3/bXd38GHP3oe4svTuRpZ50yCC7x4Ly\u002Bs=",
      "extension": {
        "E32473_extension1": "Some string applicable to a particular key",
        "E32473_extension2": {
          "property1": 10111,
          "property2": "Some text",
          "property3": {
            "subprop1": 21,
            "subprop2": true
          }
        }
      }
    }
  ],
  "initiator_sae_id": "encryptor1",
  "target_sae_ids": [
```

```

    "encryptor2"
  ],
  "ack_callback_url": "https://kme1/kmapi/v1/ext_keys/ack",
  "extension_mandatory": {
    "E32473_route_type": "direct",
    "E32473_method": {
      "hybrid_keys": true,
      "primary": "qkd",
      "secondary": "pqc"
    }
  },
  "extension_optional": {
    "E32473_extension1": "Some string applicable to all keys in the container",
    "E32473_module_type": {
      "vendor": "Company ABC",
      "protocol": "BB84",
      "min_version": 2.5
    },
    "E32473_qos_session": "e73d9abe"
  }
}

```

6.2.3 응답(Responses)

6.2.3.1 200 성공(동기 모드) 응답 application/json

동기 모드 요청에 대한 성공(OK) 응답. 선택 사항인 동기 모드를 지원하지 않는 호스트는 이 응답을 반환하지 않는다.

응답 본문은 <표 6.2.3.1-1>에 나타난 "ack_container" 스키마와 일치하는 객체의 배열인 "ack_containers" 스키마와 일치해야 하며, IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

<표 6.2.3.1-1> POST /kmapi/v1/ext_keys에 대한 200 응답의 배열 요소로 사용되는 데이터 유형 "ack_container"(application/json)

Field Name	Type	설명
key_id_container	array of object schema: key_id_container	REQUIRED 키 ID 컨테이너 "key_id_container"는 객체의 배열이어야 하며, 각 객체는 "key_id"와 선택적인 "extension" 정보를 제공한다. Maximum items: 1 024
ack_status	string schema: ack_status	REQUIRED 확인 응답 상태 "ack_status"는 확인 응답된 키의 상태를 기술하는 열거형(enum)이어야 한다. "ack_status"의 사용에 대한 세부 사항은 "ext_keys/ack"를 참조한다. Maximum length: 16
initiator_sae_id	string schema: initiator_sae_id	REQUIRED 개시 SAE ID "initiator_sae_id"는 하나 이상의 대상 SAE 와 키 공유를 최초로 요청한 SAE 의 ID 여야 한다. "ext_keys"에서 키를 참조하는 데 사용할 수 있다. 또한 "ext_keys/void"나 "ext_keys/ack"에서 이전 "ext_keys" 호출(들)의 키를 참조하는 데에도 사용할 수 있다. KME 는 (내부) SAE 로 동작하여, 역시 SAE 로 동작하는 다른 KME 와 키 공유를 요청할 수 있다. 요청의 출처에 개시 SAE 와 대상 SAE 의 구별이 적용되지 않는 경우, 구현은 임의로 하나를 "initiator_sae_id"로, 나머지를 "target_sae_id"로 지정할 수 있다. SAE ID 는 본문 4.6 절의 요구사항을 따라야 한다. Maximum length: 64
target_sae_ids	array of string schema: target_sae_ids	REQUIRED "target_sae_ids"는 요청과 관련된 대상 SAE 의 ID 인 "target_sae_id"의 배열이어야 한다. 대상은 하나 또는 여러 개를 지정할 수 있다. 개시 SAE 와 더불어 각

Field Name	Type	설명
		대상 SAE 는 키의 동일한 사본을 받을 자격이 있는 SAE 이다. KME 는 대상 SAE 를 하나만 지원(배열에 단일 SAE ID)하거나 지원하는 대상 SAE 의 최대 수를 제한할 수 있다. 가용 KME 의 능력을 초과하는 요청은 "503"('Error of server side') 응답 코드로 거부될 수 있다.
message	string schema: ack_message	확인 응답 메시지 "ack_message"는 "ack_status"를 보충하는 추가 세부 정보를 선택적으로 제공한다. Maximum length: 4 096
extension	object schema: extension	확장 "extension"은 각각 ETSI 또는 다른 조직(예: 벤더)이 규정한 확장을 나타내는 임의의 개수의 속성을 담은 객체여야 한다. KME 는 일련의 확장을 구현하도록 선택할 수 있다. 확장의 규격에는 요구사항이 포함될 수 있으며, 확장을 구현하는 KME 는 그러한 요구사항을 준수해야 한다. 추가 요구사항은 4.5 절에 제시되어 있다. Minimum properties: 1 Maximum properties: 1 024

예: "/kmapi/v1/ext_keys"에 대한 "200" 상태 코드 "ack_containers" 응답 본문의 예(미디어 유형 "application/json"):

```
[
  {
    "key_id_container": [
      {
        "key_id": "550e8400-e29b-41d4-a716-446655440000",
        "extension": {
          "E32473_extension1": "Some string applicable to a particular key",
          "E32473_extension2": {
            "property1": 10111,
            "property2": {
              "subprop1": 21,
              "subprop2": true
            }
          }
        }
      }
    ],
    "ack_status": "relayed",
    "initiator_sae_id": "encryptor1",
    "target_sae_ids": [
      "encryptor2"
    ],
    "message": "string",
    "extension": {
      "E32473_extension1": "Some string applicable to all keys in the container",
      "E32473_extension2": {
        "property1": 10111,
        "property2": {
          "subprop1": 21,
          "subprop2": true
        }
      }
    }
  }
]
```

6.2.3.2 202 요청 수락(Request accepted) 응답

서버가 요청을 수락했음을 나타내기 위해 이 응답을 보낼 수 있다. 이는 요청 수신 후 부당한 지연 없이(예: 다른 KME와의 통신 등을 기다리지 않고) 응답이 주어질 것으로 기대되는 비동기 모드 요청을 위한 것이다.

응답에는 본문이 포함되어서는 안 된다.

6.2.3.3 400 잘못된 요청 형식(Bad request format) 응답 application/json

요청 형식의 오류를 반영하는 데 사용된다. 요청 내 키와의 연관을 KME 가 유지했을 가능성이 있는 경우, KME 는 응답 코드 400 을 반환할 수 없다. 요구사항은 4.2 절을 참조한다.

응답 본문은 <표 6.2.3.3-1>에 나타난 "problem_details" 스키마와 일치해야 하며, IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

<표 6.2.3.3-1> POST /kmap/v1/ext_keys 에 대한 400 응답의 데이터 유형 "problem_details"(application/json)

Field Name	Type	설명
type	string	REQUIRED 문제 유형을 식별하는 URI 참조. Format: uri Maximum length: 1 024
status	integer	이 문제 발생에 대해 원 서버가 생성한 HTTP 상태 코드. Format: int32 Minimum: 100 Maximum: 599
title	string	사람이 읽을 수 있는 문제 유형의 짧은 요약. Maximum length: 1 024
details	object	문제에 관한 세부 정보를 전달하기 위한 하나 또는 여러 개의 문자열 속성을 담은 객체. 일반적으로 각 문자열 속성의 이름은 주제를 식별하고, 문자열 값은 더 자세한 정보를 제공한다. "details"는 문자열 사전으로 역직렬화할 수 있다. Minimum properties: 1 Maximum properties: 1 024

예 1: 요청에서 파라미터가 누락된 경우의 응답 예. "/kmap/v1/ext_keys"에 대한 "400" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```
{
  "type": "https://qkd.etsi.org/gs020-interop-kms/missing-parameters",
  "status": 400,
  "title": "missing parameters",
  "details": {
    "missing_parameters": "Required parameters are missing from the API call."
  }
}
```

예 2: "/kmap/v1/ext_keys"에 대한 "400" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```
{
  "type": "https://qkd.etsi.org/gs020-interop-kms/key-routing-error",
  "status": 400,
  "title": "key routing error",
  "details": {
    "target_sae_id_not_recognized": "KME associated with a target SAE ID is not known."
  }
}
```

예 3: "/kmap/v1/ext_keys"에 대한 "400" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```
{
  "type": "https://qkd.etsi.org/gs020-interop-kms/key-routing-error",
  "status": 400,
  "title": "key routing error",
  "details": {
```

```

    "invalid_routing": "Routing this key to target SAE ID requires directly passing it back to
the calling KME, which is invalid."
  }
}

```

6.2.3.4 401 인가 실패(Unauthorized) 응답 application/json

요청이 성공적으로 인가되지 않았음을 반영하는 데 사용된다. 요청 내 키와의 연관을 KME 가 유지했을 가능성이 있는 경우, KME 는 응답 코드 401 을 반환할 수 없다. 요구사항은 4.2 절을 참조한다.

응답 본문은 <표 6.2.3.4-1>에 나타난 "problem_details" 스키마와 일치해야 하며, IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

**<표 6.2.3.4-1> POST /kmap/v1/ext_keys 에 대한 401 응답의 데이터 유형
"problem_details"(application/json)**

Field Name	Type	설명
type	string	REQUIRED 문제 유형을 식별하는 URI 참조. Format: uri Maximum length: 1 024
status	integer	이 문제 발생에 대해 원 서버가 생성한 HTTP 상태 코드. Format: int32 Minimum: 100 Maximum: 599
title	string	사람이 읽을 수 있는 문제 유형의 짧은 요약. Maximum length: 1 024
details	object	문제에 관한 세부 정보를 전달하기 위한 하나 또는 여러 개의 문자열 속성을 담은 객체. 일반적으로 각 문자열 속성의 이름은 주제를 식별하고, 문자열 값은 더 자세한 정보를 제공한다. "details"는 문자열 사전으로 역직렬화할 수 있다. Minimum properties: 1 Maximum properties: 1 024

예: "/kmap/v1/ext_keys"에 대한 "401" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```

{
  "type": "https://qkd.etsi.org/gs020-interop-kms/unauthorized",
  "status": 401,
  "title": "unauthorized",
  "details": {
    "unauthorized": "User-supplied certificate is invalid."
  }
}

```

6.2.3.5 408 요청 타임아웃(Request timeout) 응답 application/json

요청 타임아웃 응답. 완전한 요청이 수신되지 않아 서버가 일정 시간 무활동 후 연결을 종료하는 경우에 발행될 가능성이 가장 높다.

응답 본문은 <표 6.2.3.5-1>에 나타난 "problem_details" 스키마와 일치해야 하며, IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

**<표 6.2.3.5-1> POST /kmap/v1/ext_keys 에 대한 408 응답의 데이터 유형
"problem_details"(application/json)**

Field Name	Type	설명
type	string	REQUIRED 문제 유형을 식별하는 URI 참조.

Field Name	Type	설명
		Format: uri Maximum length: 1 024
status	integer	이 문제 발생에 대해 원 서버가 생성한 HTTP 상태 코드. Format: int32 Minimum: 100 Maximum: 599
title	string	사람이 읽을 수 있는 문제 유형의 짧은 요약. Maximum length: 1 024
details	object	문제에 관한 세부 정보를 전달하기 위한 하나 또는 여러 개의 문자열 속성을 담은 객체. 일반적으로 각 문자열 속성의 이름은 주제를 식별하고, 문자열 값은 더 자세한 정보를 제공한다. "details"는 문자열 사전으로 역직렬화할 수 있다. Minimum properties: 1 Maximum properties: 1 024

예: "/kmapi/v1/ext_keys"에 대한 "408" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```
{
  "type": "https://qkd.etsi.org/gs020-interop-kms/timeout",
  "status": 408,
  "title": "timeout",
  "details": {
    "timeout": "A complete request was not received in time."
  }
}
```

6.2.3.6 503 서버 측 오류(Error of server side) 응답 application/json

응답 본문은 <표 6.2.3.6-1>에 나타난 "problem_details" 스키마와 일치해야 하며, IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

<표 6.2.3.6-1> POST /kmapi/v1/ext_keys 에 대한 503 응답의 데이터 유형 "problem_details"(application/json)

Field Name	Type	설명
type	string	REQUIRED 문제 유형을 식별하는 URI 참조. Format: uri Maximum length: 1 024
status	integer	이 문제 발생에 대해 원 서버가 생성한 HTTP 상태 코드. Format: int32 Minimum: 100 Maximum: 599
title	string	사람이 읽을 수 있는 문제 유형의 짧은 요약. Maximum length: 1 024
details	object	문제에 관한 세부 정보를 전달하기 위한 하나 또는 여러 개의 문자열 속성을 담은 객체. 일반적으로 각 문자열 속성의 이름은 주제를 식별하고, 문자열 값은 더 자세한 정보를 제공한다. "details"는 문자열 사전으로 역직렬화할 수 있다. Minimum properties: 1 Maximum properties: 1 024

예 1: "/kmapi/v1/ext_keys"에 대한 "503" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```
{
  "type": "https://qkd.etsi.org/gs020-interop-kms/server-side-general-error",
```

```

    "status": 503,
    "title": "server side general error",
    "details": {
        "server_side_general_error": "The server encountered a general failure and cannot respond."
    }
}

```

예 2: "/kmap/v1/ext_keys"에 대한 "503" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```

{
    "type": "https://qkd.etsi.org/gs020-interop-kms/key-routing-error",
    "status": 503,
    "title": "key routing error",
    "details": {
        "no_route_multiple_targets": "Routes not available to support these multiple target SAEs."
    }
}

```

예 3: "/kmap/v1/ext_keys"에 대한 "503" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```

{
    "type": "https://qkd.etsi.org/gs020-interop-kms/key-routing-error",
    "status": 503,
    "title": "key routing error",
    "details": {
        "no_known_route": "KME does not know a route to deliver to a target SAE ID."
    }
}

```

예 4: "/kmap/v1/ext_keys"에 대한 "503" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```

{
    "type": "https://qkd.etsi.org/gs020-interop-kms/insufficient-key-material",
    "status": 503,
    "title": "insufficient key material",
    "details": {
        "insufficient_key_material": "Insufficient key material available to deliver key."
    }
}

```

예 5: 요청과 함께 제공된 두 개의 "extension_mandatory" 확장을 처리할 수 없는 KME의 응답 예. "/kmap/v1/ext_keys"에 대한 "503" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```

{
    "type": "https://qkd.etsi.org/gs020-interop-kms/unsupported-mandatory-extension",
    "status": 503,
    "title": "unsupported mandatory extension",
    "details": {
        "unsupported_mandatory_extension": "Mandatory extensions E32473_ext1 and E32473_ext2 cannot be supported for this request."
    }
}

```

6.3 POST /kmapi/v1/ext_keys/ack: API 요청 후 키 상태 확인 응답

6.3.1 설명

외부 KMS 가 앞서 호출한 "ext_keys" 또는 "ext_keys/void" 메서드와 연관된 키 ID 로 구성된 키 확인 응답 컨테이너 "ack_container" 하나 이상의 배열을 전달한다. 각 컨테이너 내 모든 키의 상태는 status 필드로 나타낸다. 서로 다른 키 집합에 대해 하나 이상의 추가 상태를 보고해야 하는 경우, 하나 이상의 "ack_container"를 배열에 추가할 수 있다.

"ack_status" 필드는 다음 중 하나 이외의 값을 가져서는 안 된다: "relayed", "voided", "failed", "failed to void", "key not present".

- "relayed"는 KME 가 지정된 키를 "target_sae_ids"의 대상(들)에 성공적으로 릴레이했음을 나타낸다.
- "voided"는 KME 가 지정된 키를 성공적으로 무효화하여 애플리케이션에 전달되지 않을 것임을 나타낸다.
- "failed"는 지정된 키에 대한 "ext_keys" 릴레이 요청을 완료할 수 없었음을 나타낸다. 예를 들어 릴레이에 필요한 키 자료가 부족하면 "ext_keys" 요청이 완료되지 않을 수 있다.
- "failed to void"는 지정된 키에 대한 "ext_keys/void" 요청을 완료할 수 없었음을 나타낸다. 예를 들어 KME 가 오프라인이면 "ext_keys/void" 요청이 완료되지 않을 수 있다. 지정된 키가 이미 요청 SAE 에 전달된 경우 "ext_keys/void" 요청은 실패한다는 점에 유의한다.
- "key not present"는 KME 가 지정된 키를 찾을 수 없었음을 나타낸다.

"status" 필드 외에 추가 정보(예: 실패에 대한 설명)를 선택적인 "message" 필드에 포함할 수 있다.

하나의 "ext_keys" 요청에 여러 키가 포함되어 일부는 성공적으로 전달되고 다른 일부는 실패할 수 있다. 이는 KME 가 서로 다른 키 집합에 서로 다른 상태를 지정하기 위해 배열에 둘 이상의 확인 응답 요청 "ack_container"를 포함함으로써 나타낼 수 있다. 예를 들어 일부 키에 대해 다른 키보다 더 빠른 확인 응답을 가능하게 하기 위해 "ext_keys/ack"를 여러 번 호출할 수도 있다.

"ext_keys/ack" 호출에서 확인 응답되는 키가 반드시 "ext_keys" 또는 "ext_keys/void" 메서드에 대한 단일 요청에서 비롯되는 것은 아니다. 키는 어떤 방식으로든 조합될 수 있으나, 배열 내 각 키 확인 응답 컨테이너 "ack_container"는 동일한 "ack_status", "initiator_sae_id" 및 "target_sae_ids"를 공유하는 확인 응답을 묶는다.

역자 주 - 원문은 여기서 "status" 필드, "message" 필드라고 적고 있으나, 스키마상 필드명은 각각 "ack_status"와 "message"(스키마 ack_message)이다.

확장 필드도 선택적으로 포함할 수 있다(예: 원격 KMS 로부터의 라우팅 정보를 나타내는 각 키의 메타데이터). 이러한 확장이 포함된 경우, 확장을 그에 맞게 처리하는 것이 좋다(예: 개시 SAE 에 릴레이할 수 있도록 제공된 메타데이터를 각 키와 함께 보관).

유효한 확인 응답을 받은 KME 는 "200" 코드를 반환해야 하며, 그렇지 않으면 오류의 세부 사항을 포함하여 오류를 나타내는 상태 코드를 반환해야 한다.

6.3.2 요청 본문(Request Body): application/json

요청 본문은 <표 6.3.2-1>에 나타낸 "ack_container" 스키마와 일치하는 객체의 배열인 "ack_containers" 스키마와 일치해야 하며, IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

**<표 6.3.2-1> POST /kmapi/v1/ext_keys/ack 요청의 배열 요소로 사용되는 데이터 유형
"ack_container"(application/json)**

Field Name	Type	설명
key_id_container	array of object schema: key_id_container	REQUIRED 키 ID 컨테이너 "key_id_container"는 객체의 배열이어야 하며, 각 객체는 "key_id"와 선택적인 "extension" 정보를 제공한다. Maximum items: 1 024

Field Name	Type	설명
ack_status	string schema: ack_status	REQUIRED 확인 응답 상태 "ack_status"는 확인 응답된 키의 상태를 기술하는 열거형(enum)이어야 한다. "ack_status"의 사용에 대한 세부 사항은 "ext_keys/ack"를 참조한다. Maximum length: 16
initiator_sae_id	string schema: initiator_sae_id	REQUIRED 개시 SAE ID "initiator_sae_id"는 하나 이상의 대상 SAE 와 키 공유를 최초로 요청한 SAE 의 ID 여야 한다. "ext_keys"에서 키를 참조하는 데 사용할 수 있다. 또한 "ext_keys/void"나 "ext_keys/ack"에서 이전 "ext_keys" 호출(들)의 키를 참조하는 데에도 사용할 수 있다. KME 는 (내부) SAE 로 동작하여, 역시 SAE 로 동작하는 다른 KME 와 키 공유를 요청할 수 있다. 요청의 출처에 개시 SAE 와 대상 SAE 의 구별이 적용되지 않는 경우, 구현은 임의로 하나를 "initiator_sae_id"로, 나머지를 "target_sae_id"로 지정할 수 있다. SAE ID 는 본문 4.6 절의 요구사항을 따라야 한다. Maximum length: 64
target_sae_ids	array of string schema: target_sae_ids	REQUIRED "target_sae_ids"는 요청과 관련된 대상 SAE 의 ID 인 "target_sae_id"의 배열이어야 한다. 대상은 하나 또는 여러 개를 지정할 수 있다. 개시 SAE 와 더불어 각 대상 SAE 는 키의 동일한 사본을 받을 자격이 있는 SAE 이다. KME 는 대상 SAE 를 하나만 지원(배열에 단일 SAE ID)하거나 지원하는 대상 SAE 의 최대 수를 제한할 수 있다. 가용 KME 의 능력을 초과하는 요청은 "503"('Error of server side') 응답 코드로 거부될 수 있다.
message	string schema: ack_message	확인 응답 메시지 "ack_message"는 "ack_status"를 보충하는 추가 세부 정보를 선택적으로 제공한다. Maximum length: 4 096
extension	object schema: extension	확장 "extension"은 각각 ETSI 또는 다른 조직(예: 벤더)이 규정한 확장을 나타내는 임의 개수의 속성을 담은 객체여야 한다. KME 는 일련의 확장을 구현하도록 선택할 수 있다. 확장의 규격에는 요구사항이 포함될 수 있으며, 확장을 구현하는 KME 는 그러한 요구사항을 준수해야 한다. 추가 요구사항은 4.5 절에 제시되어 있다. Minimum properties: 1 Maximum properties: 1 024

예: "/kmapi/v1/ext_keys/ack"에 대한 "ack_containers" 요청 본문의 예(미디어 유형 "application/json"):

```
[
  {
    "key_id_container": [
      {
        "key_id": "550e8400-e29b-41d4-a716-446655440000",
        "extension": {
          "E32473_extension1": "Some string applicable to a particular key",
          "E32473_extension2": {
            "property1": 10111,
            "property2": {
              "subprop1": 21,
              "subprop2": true
            }
          }
        }
      }
    ]
  },
  "ack_status": "relayed",
  "initiator_sae_id": "encryptor1",
  "target_sae_ids": [
    "encryptor2"
  ],
  "message": "string",
```

```

    "extension": {
      "E32473_extension1": "Some string applicable to all keys in the container",
      "E32473_extension2": {
        "property1": 10111,
        "property2": {
          "subprop1": 21,
          "subprop2": true
        }
      }
    }
  }
}
]

```

6.3.3 응답(Responses)

6.3.3.1 200 성공(Successful OK) 응답

응답에는 본문이 포함되어서는 안 된다.

6.3.3.2 400 잘못된 요청 형식(Bad request format) 응답 application/json

응답 본문은 <표 6.3.3.2-1>에 나타난 "problem_details" 스키마와 일치해야 하며, IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

<표 6.3.3.2-1> POST /kmapi/v1/ext_keys/ack 에 대한 400 응답의 데이터 유형 "problem_details"(application/json)

Field Name	Type	설명
type	string	REQUIRED 문제 유형을 식별하는 URI 참조. Format: uri Maximum length: 1 024
status	integer	이 문제 발생에 대해 원 서버가 생성한 HTTP 상태 코드. Format: int32 Minimum: 100 Maximum: 599
title	string	사람이 읽을 수 있는 문제 유형의 짧은 요약. Maximum length: 1 024
details	object	문제에 관한 세부 정보를 전달하기 위한 하나 또는 여러 개의 문자열 속성을 담은 객체. 일반적으로 각 문자열 속성의 이름은 주제를 식별하고, 문자열 값은 더 자세한 정보를 제공한다. "details"는 문자열 사전으로 역직렬화할 수 있다. Minimum properties: 1 Maximum properties: 1 024

예 1: "/kmapi/v1/ext_keys/ack"에 대한 "400" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```

{
  "type": "https://qkd.etsi.org/gs020-interop-kms/ack-not-understood",
  "status": 400,
  "title": "ack not understood",
  "details": {
    "ack_not_understood": "KME cannot parse the submitted acknowledgement."
  }
}

```

예 2: "/kmapi/v1/ext_keys/ack"에 대한 "400" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```

{

```

```

    "type": "https://qkd.etsi.org/gs020-interop-kms/key-not-present",
    "title": "key not present",
    "details": {
      "key_not_present": "The acknowledged key ID(s) are not known to this KME."
    }
  }
}

```

6.3.3.3 401 인가 실패(Unauthorized) 응답 application/json

응답 본문은 <표 6.3.3.3-1>에 나타난 "problem_details" 스키마와 일치해야 하며, IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

<표 6.3.3.3-1> POST /kmapi/v1/ext_keys/ack 에 대한 401 응답의 데이터 유형 "problem_details"(application/json)

Field Name	Type	설명
type	string	REQUIRED 문제 유형을 식별하는 URI 참조. Format: uri Maximum length: 1 024
status	integer	이 문제 발생에 대해 원 서버가 생성한 HTTP 상태 코드. Format: int32 Minimum: 100 Maximum: 599
title	string	사람이 읽을 수 있는 문제 유형의 짧은 요약. Maximum length: 1 024
details	object	문제에 관한 세부 정보를 전달하기 위한 하나 또는 여러 개의 문자열 속성을 담은 객체. 일반적으로 각 문자열 속성의 이름은 주제를 식별하고, 문자열 값은 더 자세한 정보를 제공한다. "details"는 문자열 사전으로 역직렬화할 수 있다. Minimum properties: 1 Maximum properties: 1 024

예: "/kmapi/v1/ext_keys/ack"에 대한 "401" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```

{
  "type": "https://qkd.etsi.org/gs020-interop-kms/unauthorized",
  "status": 401,
  "title": "unauthorized",
  "details": {
    "unauthorized": "User-supplied certificate is invalid."
  }
}

```

6.3.3.4 503 서버 측 오류(Error of server side) 응답 application/json

응답 본문은 <표 6.3.3.4-1>에 나타난 "problem_details" 스키마와 일치해야 하며, IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

<표 6.3.3.4-1> POST /kmapi/v1/ext_keys/ack 에 대한 503 응답의 데이터 유형 "problem_details"(application/json)

Field Name	Type	설명
type	string	REQUIRED 문제 유형을 식별하는 URI 참조. Format: uri Maximum length: 1 024

Field Name	Type	설명
status	integer	이 문제 발생에 대해 원 서버가 생성한 HTTP 상태 코드. Format: int32 Minimum: 100 Maximum: 599
title	string	사람이 읽을 수 있는 문제 유형의 짧은 요약. Maximum length: 1 024
details	object	문제에 관한 세부 정보를 전달하기 위한 하나 또는 여러 개의 문자열 속성을 담은 객체. 일반적으로 각 문자열 속성의 이름은 주제를 식별하고, 문자열 값은 더 자세한 정보를 제공한다. "details"는 문자열 사전으로 역직렬화할 수 있다. Minimum properties: 1 Maximum properties: 1 024

예 1: "/kmapi/v1/ext_keys/ack"에 대한 "503" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```
{
  "type": "https://qkd.etsi.org/gs020-interop-kms/server-side-general-error",
  "status": 503,
  "title": "server side general error",
  "details": {
    "server_side_general_error": "The server encountered a general failure and cannot respond."
  }
}
```

예 2: "/kmapi/v1/ext_keys/ack"에 대한 "503" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```
{
  "type": "https://qkd.etsi.org/gs020-interop-kms/key-routing-error",
  "status": 503,
  "title": "key routing error",
  "details": {
    "no_route_multiple_targets": "Routes not available to support these multiple target SAEs."
  }
}
```

6.4 POST /kmapi/v1/ext_keys/void: 외부 KMS 에 키 무효화 신호(즉, 키 폐기)

6.4.1 설명

키 ID 로 구성된 무효화 요청 컨테이너 "void_container"를 다른 KME 에 전달하여, 키가 무효(즉, 폐기되어 SAE 에 전달되지 않음)로 표시되도록 한다. 무효화 요청 컨테이너 "void_container"에는 이미 해당 KME 로 전달된 키와 일치하는 키가 담긴다.

유효한 요청을 받으면, 필수 비동기 모드의 경우 KME 는 제공된 키 ID 와 관련된 키를 폐기하고 완료된 동작을 기술하는 호출을 지정된 "ack_callback_url"로 보내야 한다. 이후 KME 는 영향을 받은 키를 사용해서는 안 되며, 해당 키를 가져가려는 모든 애플리케이션 요청을 거부하는 것도 포함된다.

비동기 모드에 더하여 동기 모드를 구현한 경우, KME 는 동기 모드 요청(이러한 요청에서는 "ack_callback_url"이 생략되었을 것이다)의 키에 대한 결과를 확인 응답하기 위해 "ext_keys/ack"를 사용해서는 안 된다. 대신 KME 는 "200"('Successful OK')으로 응답해야 한다.

추가 조사 없이도 요청이 유효하지 않음을 아는 경우 "400" 코드를 반환해야 한다. 그렇지 않으면 키 무효화 실패는 이후 "ack_callback_url"을 통해 보고할 수 있다.

이 연산이 빈 "key_ids" 배열로 요청되면, 미리 지정된 SAE 간에 공유된 모든 키를 무효화해야 한다. 의도하지 않은 키 손실 위험을 줄이기 위해, KME 는 "void_container"에 값이 true 인 "all_confirmation" 파라미터가 포함되지

않는 한 하나 이상의 "key_id"를 지정하지 않은 무효화 요청을 거부해야 한다. 이러한 거부의 경우 KME 는 "400" 코드를 반환해야 한다.

6.4.2 요청 본문(Request Body): application/json

요청 본문은 <표 6.4.2-1>에 나타낸 "void_container" 스키마와 일치해야 하며, IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

<표 6.4.2-1> POST /kmap/v1/ext_keys/void 요청의 데이터 유형 "void_container"(application/json)

Field Name	Type	설명
key_ids	array of string schema: key_ids	REQUIRED 키 ID "key_ids"는 "key_id"의 배열이어야 한다.
initiator_sae_id	string schema: initiator_sae_id	REQUIRED 개시 SAE ID "initiator_sae_id"는 하나 이상의 대상 SAE 와 키 공유를 최초로 요청한 SAE 의 ID 여야 한다. "ext_keys"에서 키를 참조하는 데 사용할 수 있다. 또한 "ext_keys/void"나 "ext_keys/ack"에서 이전 "ext_keys" 호출(들)의 키를 참조하는 데에도 사용할 수 있다. KME 는 (내부) SAE 로 동작하여, 역시 SAE 로 동작하는 다른 KME 와 키 공유를 요청할 수 있다. 요청의 출처에 개시 SAE 와 대상 SAE 의 구별이 적용되지 않는 경우, 구현은 임의로 하나를 "initiator_sae_id"로, 나머지를 "target_sae_id"로 지정할 수 있다. SAE ID 는 본문 4.6 절의 요구사항을 따라야 한다. Maximum length: 64
target_sae_ids	array of string schema: target_sae_ids	REQUIRED "target_sae_ids"는 요청과 관련된 대상 SAE 의 ID 인 "target_sae_id"의 배열이어야 한다. 대상은 하나 또는 여러 개를 지정할 수 있다. 개시 SAE 와 더불어 각 대상 SAE 는 키의 동일한 사본을 받을 자격이 있는 SAE 이다. KME 는 대상 SAE 를 하나만 지원(배열에 단일 SAE ID)하거나 지원하는 대상 SAE 의 최대 수를 제한할 수 있다. 가용 KME 의 능력을 초과하는 요청은 "503"('Error of server side') 응답 코드로 거부될 수 있다.
ack_callback_url	string schema: ack_callback_url	확인 응답 콜백 URL "ack_callback_url"은 요청의 전부 또는 일부가 완료되거나 실패한 후 확인 응답(들)을 보낼 URL 이다. 필수인 비동기 모드에서는 이 파라미터가 필수이며 지정되어야 한다. 이 파라미터가 생략되면 요청은 선택 사항인 동기 모드로 엔드포인트를 호출하는 요청으로 해석되어야 한다. 그렇지 않으면 필수인 비동기 모드로 엔드포인트를 호출하는 요청으로 해석되어야 한다. 필수 비동기 모드와 선택 동기 모드의 동작은 4.2 절을 참조한다. 새로운 KME 에 "ack_callback_url" 없이 동기 모드 요청을 보내기 전에, "/kmap/versions" 엔드포인트를 호출하여 새 KME 가 선택 사항인 동기 모드 "capability"를 지원하는지 확인하는 것이 좋다. "ack_callback_url"을 제공하는 경우, 프로토콜로 시작하는, 즉 "https://"로 시작하는 완전한 URL 이어야 한다. authority 에는 도메인이나 IP 주소 외에 선택적으로 포트 번호를 포함할 수 있다. Format: uri Maximum length: 1 024
extension	object schema: extension	확장 "extension"은 각각 ETSI 또는 다른 조직(예: 벤더)이 규정한 확장을 나타내는 임의 개수의 속성을 담은 객체여야 한다. KME 는 일련의 확장을 구현하도록 선택할 수 있다. 확장의 규격에는 요구사항이 포함될 수 있으며, 확장을 구현하는 KME 는 그러한 요구사항을 준수해야 한다. 추가 요구사항은 4.5 절에 제시되어 있다. Minimum properties: 1 Maximum properties: 1 024
all_confirmation	boolean schema:	"all_confirmation"은 확인 플래그 역할을 하며, "ext_keys/void" 호출에서 키 ID 가 지정되지 않았을 때 지정된 SAE 간의 모든 키를 무효화하려는 의도임을 확인하기

Field Name	Type	설명
	all_confirmation	위해 "true" 값과 함께 포함되어야 한다. "ext_keys/void" 호출에서 키 ID 가 지정되지 않았는데 "all_confirmation"이 생략되었거나 그 값이 "true"가 아니면, 응답은 "details" 안에 "no_all_confirmation"이라는 문자열을 포함한 "400"('Bad request format') 이어야 한다.

예: "/kmap/v1/ext_keys/void"에 대한 "void_container" 요청 본문의 예(미디어 유형 "application/json"):

```
{
  "key_ids": [
    "550e8400-e29b-41d4-a716-446655440000",
    "373b0b2c-d841-4765-af6f-c6232cda6531",
    "65380d2d-7de4-4445-9a1e-23f77218e61d"
  ],
  "initiator_sae_id": "encryptor1",
  "target_sae_ids": [
    "encryptor2"
  ],
  "ack_callback_url": "https://kme1/kmap/v1/ext_keys/ack",
  "extension": {
    "E32473_extension1": "Some string",
    "E32473_extension2": {
      "property1": 10111,
      "property2": "Some text",
      "property3": {
        "subprop1": 21,
        "subprop2": true
      }
    }
  }
}
```

6.4.3 응답(Responses)

6.4.3.1 200 성공(동기 모드) 응답 application/json

동기 모드 요청에 대한 성공(OK) 응답. 선택 사항인 동기 모드를 지원하지 않는 호스트는 이 응답을 반환하지 않는다.

응답 본문은 <표 6.4.3.1-1>에 나타낸 "ack_container" 스키마와 일치하는 객체의 배열인 "ack_containers" 스키마와 일치해야 하며, IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

<표 6.4.3.1-1> POST /kmap/v1/ext_keys/void 에 대한 200 응답의 배열 요소로 사용되는 데이터 유형 "ack_container"(application/json)

Field Name	Type	설명
key_id_container	array of object schema: key_id_container	REQUIRED 키 ID 컨테이너 "key_id_container"는 객체의 배열이어야 하며, 각 객체는 "key_id"와 선택적인 "extension" 정보를 제공한다. Maximum items: 1 024
ack_status	string schema: ack_status	REQUIRED 확인 응답 상태 "ack_status"는 확인 응답된 키의 상태를 기술하는 열거형(enum) 이어야 한다. "ack_status"의 사용에 대한 세부 사항은 "ext_keys/ack"를 참조한다. Maximum length: 16
initiator_sae_id	string schema: initiator_sae_id	REQUIRED 개시 SAE ID "initiator_sae_id"는 하나 이상의 대상 SAE 와 키 공유를 최초로 요청한 SAE 의 ID 여야 한다. "ext_keys"에서 키를 참조하는 데 사용할 수 있다. 또한 "ext_keys/void"나 "ext_keys/ack"에서 이전 "ext_keys" 호출(들)의 키를

Field Name	Type	설명
		참조하는 데에도 사용할 수 있다. KME 는 (내부) SAE 로 동작하여, 역시 SAE 로 동작하는 다른 KME 와 키 공유를 요청할 수 있다. 요청의 출처에 개시 SAE 와 대상 SAE 의 구별이 적용되지 않는 경우, 구현은 임의로 하나를 "initiator_sae_id"로, 나머지를 "target_sae_id"로 지정할 수 있다. SAE ID 는 본문 4.6 절의 요구사항을 따라야 한다. Maximum length: 64
target_sae_ids	array of string schema: target_sae_ids	REQUIRED "target_sae_ids"는 요청과 관련된 대상 SAE 의 ID 인 "target_sae_id"의 배열이어야 한다. 대상은 하나 또는 여러 개를 지정할 수 있다. 개시 SAE 와 더불어 각 대상 SAE 는 키의 동일한 사본을 받을 자격이 있는 SAE 이다. KME 는 대상 SAE 를 하나만 지원(배열에 단일 SAE ID)하거나 지원하는 대상 SAE 의 최대 수를 제한할 수 있다. 가용 KME 의 능력을 초과하는 요청은 "503"('Error of server side') 응답 코드로 거부될 수 있다.
message	string schema: ack_message	확인 응답 메시지 "ack_message"는 "ack_status"를 보충하는 추가 세부 정보를 선택적으로 제공한다. Maximum length: 4 096
extension	object schema: extension	확장 "extension"은 각각 ETSI 또는 다른 조직(예: 벤더)이 규정한 확장을 나타내는 임의의 개수의 속성을 담은 객체여야 한다. KME 는 일련의 확장을 구현하도록 선택할 수 있다. 확장의 규격에는 요구사항이 포함될 수 있으며, 확장을 구현하는 KME 는 그러한 요구사항을 준수해야 한다. 추가 요구사항은 4.5 절에 제시되어 있다. Minimum properties: 1 Maximum properties: 1 024

예: "/kmapi/v1/ext_keys/void"에 대한 "200" 상태 코드 "ack_containers" 응답 본문의 예(미디어 유형 "application/json"):

```
[
  {
    "key_id_container": [
      {
        "key_id": "550e8400-e29b-41d4-a716-446655440000",
        "extension": {
          "E32473_extension1": "Some string applicable to a particular key",
          "E32473_extension2": {
            "property1": 10111,
            "property2": {
              "subprop1": 21,
              "subprop2": true
            }
          }
        }
      }
    ]
  },
  "ack_status": "voided",
  "initiator_sae_id": "encryptor1",
  "target_sae_ids": [
    "encryptor2"
  ],
  "message": "string",
  "extension": {
    "E32473_extension1": "Some string applicable to all keys in the container",
    "E32473_extension2": {
      "property1": 10111,
      "property2": {
        "subprop1": 21,
        "subprop2": true
      }
    }
  }
]
```

```

    }
  }
}

```

6.4.3.2 202 요청 수락(Request accepted) 응답

서버가 요청을 수락했음을 나타내기 위해 이 응답을 보낼 수 있다. 이는 요청 수신 후 부당한 지연 없이(예: 다른 KME와의 통신 등을 기다리지 않고) 응답이 주어질 것으로 기대되는 비동기 모드 요청을 위한 것이다.

응답에는 본문이 포함되어서는 안 된다.

6.4.3.3 400 잘못된 요청 형식(Bad request format) 응답 application/json

응답 본문은 <표 6.4.3.3-1>에 나타난 "problem_details" 스키마와 일치해야 하며, IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

<표 6.4.3.3-1> POST /kmap/v1/ext_keys/void 에 대한 400 응답의 데이터 유형 "problem_details"(application/json)

Field Name	Type	설명
type	string	REQUIRED 문제 유형을 식별하는 URI 참조. Format: uri Maximum length: 1 024
status	integer	이 문제 발생에 대해 원 서버가 생성한 HTTP 상태 코드. Format: int32 Minimum: 100 Maximum: 599
title	string	사람이 읽을 수 있는 문제 유형의 짧은 요약. Maximum length: 1 024
details	object	문제에 관한 세부 정보를 전달하기 위한 하나 또는 여러 개의 문자열 속성을 담은 객체. 일반적으로 각 문자열 속성의 이름은 주제를 식별하고, 문자열 값은 더 자세한 정보를 제공한다. "details"는 문자열 사전으로 역직렬화할 수 있다. Minimum properties: 1 Maximum properties: 1 024

예 1: "/kmap/v1/ext_keys/void"에 대한 "400" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```

{
  "type": "https://qkd.etsi.org/gs020-interop-kms/void-request-not-understood",
  "status": 400,
  "title": "void request not understood",
  "details": {
    "key_voiding_error": "KME cannot parse the submitted void request."
  }
}

```

예 2: "/kmap/v1/ext_keys/void"에 대한 "400" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```

{
  "type": "https://qkd.etsi.org/gs020-interop-kms/no-key-ids-or-confirmation",
  "title": "no key ids or confirmation",
  "details": {
    "no_key_ids_or_confirmation": "When no key_ids are passed, all keys shared between the SAEs will be voided. If this is the intended action, the all_confirmation field also needs to be set with a value of true. Otherwise, please specify key_ids to be voided."
  }
}

```

역자 주 – 6.3.3.2 절의 예 2와 6.4.3.3 절의 예 2는 원문에서도 "status" 필드가 빠져 있어 그대로 두었다. 또한 6.4.3.3 절 예 2의 details 키는 "no_key_ids_or_confirmation"인데, 7.6 절과 6.4.2 절 표는 이 경우의 details 문자열 이름을 "no_all_confirmation"으로 규정하고 있다.

6.4.3.4 401 인가 실패(Unauthorized) 응답 application/json

응답 본문은 <표 6.4.3.4-1>에 나타난 "problem_details" 스키마와 일치해야 하며, IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

**<표 6.4.3.4-1> POST /kmapi/v1/ext_keys/void 에 대한 401 응답의 데이터 유형
"problem_details"(application/json)**

Field Name	Type	설명
type	string	REQUIRED 문제 유형을 식별하는 URI 참조. Format: uri Maximum length: 1 024
status	integer	이 문제 발생에 대해 원 서버가 생성한 HTTP 상태 코드. Format: int32 Minimum: 100 Maximum: 599
title	string	사람이 읽을 수 있는 문제 유형의 짧은 요약. Maximum length: 1 024
details	object	문제에 관한 세부 정보를 전달하기 위한 하나 또는 여러 개의 문자열 속성을 담은 객체. 일반적으로 각 문자열 속성의 이름은 주제를 식별하고, 문자열 값은 더 자세한 정보를 제공한다. "details"는 문자열 사전으로 역직렬화할 수 있다. Minimum properties: 1 Maximum properties: 1 024

예: "/kmapi/v1/ext_keys/void"에 대한 "401" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```
{
  "type": "https://qkd.etsi.org/gs020-interop-kms/unauthorized",
  "status": 401,
  "title": "unauthorized",
  "details": {
    "unauthorized": "User-supplied certificate is invalid."
  }
}
```

6.4.3.5 408 요청 타임아웃(Request timeout) 응답 application/json

요청 타임아웃 응답. 완전한 요청이 수신되지 않아 서버가 일정 시간 무활동 후 연결을 종료하는 경우에 발행될 가능성이 가장 높다.

응답 본문은 <표 6.4.3.5-1>에 나타난 "problem_details" 스키마와 일치해야 하며, IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

**<표 6.4.3.5-1> POST /kmapi/v1/ext_keys/void 에 대한 408 응답의 데이터 유형
"problem_details"(application/json)**

Field Name	Type	설명
type	string	REQUIRED 문제 유형을 식별하는 URI 참조. Format: uri

Field Name	Type	설명
		Maximum length: 1 024
status	integer	이 문제 발생에 대해 원 서버가 생성한 HTTP 상태 코드. Format: int32 Minimum: 100 Maximum: 599
title	string	사람이 읽을 수 있는 문제 유형의 짧은 요약. Maximum length: 1 024
details	object	문제에 관한 세부 정보를 전달하기 위한 하나 또는 여러 개의 문자열 속성을 담은 객체. 일반적으로 각 문자열 속성의 이름은 주제를 식별하고, 문자열 값은 더 자세한 정보를 제공한다. "details"는 문자열 사전으로 역직렬화할 수 있다. Minimum properties: 1 Maximum properties: 1 024

예: "/kmapi/v1/ext_keys/void"에 대한 "408" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```
{
  "type": "https://qkd.etsi.org/gs020-interop-kms/timeout",
  "status": 408,
  "title": "timeout",
  "details": {
    "timeout": "A complete request was not received in time."
  }
}
```

6.4.3.6 503 서버 측 오류(Error of server side) 응답 application/json

응답 본문은 <표 6.4.3.6-1>에 나타난 "problem_details" 스키마와 일치해야 하며, IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

<표 6.4.3.6-1> POST /kmapi/v1/ext_keys/void 에 대한 503 응답의 데이터 유형 "problem_details"(application/json)

Field Name	Type	설명
type	string	REQUIRED 문제 유형을 식별하는 URI 참조. Format: uri Maximum length: 1 024
status	integer	이 문제 발생에 대해 원 서버가 생성한 HTTP 상태 코드. Format: int32 Minimum: 100 Maximum: 599
title	string	사람이 읽을 수 있는 문제 유형의 짧은 요약. Maximum length: 1 024
details	object	문제에 관한 세부 정보를 전달하기 위한 하나 또는 여러 개의 문자열 속성을 담은 객체. 일반적으로 각 문자열 속성의 이름은 주제를 식별하고, 문자열 값은 더 자세한 정보를 제공한다. "details"는 문자열 사전으로 역직렬화할 수 있다. Minimum properties: 1 Maximum properties: 1 024

예 1: "/kmapi/v1/ext_keys/void"에 대한 "503" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```
{
  "type": "https://qkd.etsi.org/gs020-interop-kms/server-side-general-error",
```

```

    "status": 503,
    "title": "server side general error",
    "details": {
      "server_side_general_error": "The server encountered a general failure and cannot
respond."
    }
  }
}

```

예 2: "/kmap/v1/ext_keys/void"에 대한 "503" 상태 코드 "problem_details" 응답 본문의 예(미디어 유형 "application/json"):

```

{
  "type": "https://qkd.etsi.org/gs020-interop-kms/key-routing-error",
  "status": 503,
  "title": "key routing error",
  "details": {
    "no_route_multiple_targets": "Routes not available to support these multiple target SAEs."
  }
}

```

7 스키마(Schemas)

7.1 ack_callback_url

확인 응답 콜백 URL "ack_callback_url"은 요청의 전부 또는 일부가 완료되거나 실패한 후 확인 응답 (들)을 보낼 URL이다.

필수인 비동기 모드에서는 이 파라미터가 필수이며 지정되어야 한다.

이 파라미터가 생략되면 요청은 선택 사항인 동기 모드로 엔드포인트를 호출하는 요청으로 해석되어야 한다. 그렇지 않으면 필수인 비동기 모드로 엔드포인트를 호출하는 요청으로 해석되어야 한다. 필수 비동기 모드와 선택 동기 모드의 동작은 4.2 절을 참조한다.

새로운 KME 에 "ack_callback_url" 없이 동기 모드 요청을 보내기 전에, "/kmap/v1/versions" 엔드포인트를 호출하여 새 KME 가 선택 사항인 동기 모드 "capability"를 지원하는지 확인하는 것이 좋다.

"ack_callback_url"을 제공하는 경우, 프로토콜로 시작하는, 즉 "https://"로 시작하는 완전한 URL 이어야 한다. authority 에는 도메인이나 IP 주소 외에 선택적으로 포트 번호를 포함할 수 있다.

데이터 형식은 string 이어야 한다.

Format: uri

Maximum length: 1 024

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "ack_callback_url"의 예:

```
"https://kme1/kmap/v1/ext_keys/ack"
```

7.2 ack_container

확인 응답 컨테이너 "ack_container"는 요청된 연산이 완료된 후 결과를 보고하는 데 사용되는 컨테이너이다.

데이터 형식은 <표 7.2-1>에 규정된 대로 IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

<표 7.2-1> "ack_container"의 데이터 유형

Field Name	Type	설명
key_id_container	array of object	REQUIRED

Field Name	Type	설명
	schema: key_id_container	키 ID 컨테이너 "key_id_container"는 객체의 배열이어야 하며, 각 객체는 "key_id"와 선택적인 "extension" 정보를 제공한다. Maximum items: 1 024
ack_status	string schema: ack_status	REQUIRED 확인 응답 상태 "ack_status"는 확인 응답된 키의 상태를 기술하는 열거형(enum)이어야 한다. "ack_status"의 사용에 대한 세부 사항은 "ext_keys/ack"를 참조한다. Maximum length: 16
initiator_sae_id	string schema: initiator_sae_id	REQUIRED 개시 SAE ID "initiator_sae_id"는 하나 이상의 대상 SAE 와 키 공유를 최초로 요청한 SAE 의 ID 여야 한다. "ext_keys"에서 키를 참조하는 데 사용할 수 있다. 또한 "ext_keys/void"나 "ext_keys/ack"에서 이전 "ext_keys" 호출(들)의 키를 참조하는 데에도 사용할 수 있다. KME 는 (내부) SAE 로 동작하여, 역시 SAE 로 동작하는 다른 KME 와 키 공유를 요청할 수 있다. 요청의 출처에 개시 SAE 와 대상 SAE 의 구별이 적용되지 않는 경우, 구현은 임의로 하나를 "initiator_sae_id"로, 나머지를 "target_sae_id"로 지정할 수 있다. SAE ID 는 본문 4.6 절의 요구사항을 따라야 한다. Maximum length: 64
target_sae_ids	array of string schema: target_sae_ids	REQUIRED "target_sae_ids"는 요청과 관련된 대상 SAE 의 ID 인 "target_sae_id"의 배열이어야 한다. 대상은 하나 또는 여러 개를 지정할 수 있다. 개시 SAE 와 더불어 각 대상 SAE 는 키의 동일한 사본을 받을 자격이 있는 SAE 이다. KME 는 대상 SAE 를 하나만 지원(배열에 단일 SAE ID)하거나 지원하는 대상 SAE 의 최대 수를 제한할 수 있다. 가용 KME 의 능력을 초과하는 요청은 "503"('Error of server side') 응답 코드로 거부될 수 있다.
message	string schema: ack_message	확인 응답 메시지 "ack_message"는 "ack_status"를 보충하는 추가 세부 정보를 선택적으로 제공한다. Maximum length: 4 096
extension	object schema: extension	확장 "extension"은 각각 ETSI 또는 다른 조직(예: 벤더)이 규정한 확장을 나타내는 임의 개수의 속성을 담은 객체여야 한다. KME 는 일련의 확장을 구현하도록 선택할 수 있다. 확장의 규격에는 요구사항이 포함될 수 있으며, 확장을 구현하는 KME 는 그러한 요구사항을 준수해야 한다. 추가 요구사항은 4.5 절에 제시되어 있다. Minimum properties: 1 Maximum properties: 1 024

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "ack_container"의 예:

```
{
  "key_id_container": [
    {
      "key_id": "550e8400-e29b-41d4-a716-446655440000",
      "extension": {
        "E32473_extension1": "Some string applicable to a particular key",
        "E32473_extension2": {
          "property1": 10111,
          "property2": {
            "subprop1": 21,
            "subprop2": true
          }
        }
      }
    }
  ],
  "ack_status": "relayed",
  "initiator_sae_id": "encryptor1",
  "target_sae_ids": [
```

```

    "encryptor2"
  ],
  "message": "string",
  "extension": {
    "E32473_extension1": "Some string applicable to all keys in the container",
    "E32473_extension2": {
      "property1": 10111,
      "property2": {
        "subprop1": 21,
        "subprop2": true
      }
    }
  }
}

```

7.3 ack_containers

확인 응답 컨테이너 "ack_containers"는 확인 응답 컨테이너 "ack_container"의 배열이어야 한다. "ack_containers"는 서로 다른 상태, SAE ID 등을 갖는 키를 확인 응답하는 데 사용할 수 있으며, 하나의 "ack_container" 안의 키들은 몇 가지 공통 속성을 공유한다.

데이터 형식은 array여야 한다.

Maximum items: 1 024

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "ack_containers"의 예:

```

[
  {
    "key_id_container": [
      {
        "key_id": "550e8400-e29b-41d4-a716-446655440000",
        "extension": {
          "E32473_extension1": "Some string applicable to a particular key",
          "E32473_extension2": {
            "property1": 10111,
            "property2": {
              "subprop1": 21,
              "subprop2": true
            }
          }
        }
      }
    ]
  },
  {
    "ack_status": "relayed",
    "initiator_sae_id": "encryptor1",
    "target_sae_ids": [
      "encryptor2"
    ],
    "message": "string",
    "extension": {
      "E32473_extension1": "Some string applicable to all keys in the container",
      "E32473_extension2": {
        "property1": 10111,
        "property2": {
          "subprop1": 21,
          "subprop2": true
        }
      }
    }
  }
]

```

7.4 ack_message

확인 응답 메시지 "ack_message"는 "ack_status"를 보충하는 추가 세부 정보를 선택적으로 제공한다.

데이터 형식은 string 이어야 한다.

Maximum length: 4 096

7.5 ack_status

확인 응답 상태 "ack_status"는 확인 응답된 키의 상태를 기술하는 열거형 (enum)이어야 한다. "ack_status"의 사용에 대한 세부 사항은 "ext_keys/ack"를 참조한다.

"ack_status"는 다음 중 하나 이외의 값을 가져서는 안 된다: "relayed", "failed", "voided", "failed to void", "key not present".

데이터 형식은 string 이어야 한다.

Maximum length: 16

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "ack_status"의 예:

```
"relayed"
```

7.6 all_confirmation

"all_confirmation"은 확인 플래그 역할을 하며, "ext_keys/void" 호출에서 키 ID 가 지정되지 않았을 때 지정된 SAE 간의 모든 키를 무효화하려는 의도임을 확인하기 위해 "true" 값과 함께 포함되어야 한다.

"ext_keys/void" 호출에서 키 ID 가 지정되지 않았는데 "all_confirmation"이 생략되었거나 그 값이 "true"가 아니면, 응답은 "details" 안에 "no_all_confirmation"이라는 문자열을 포함한 "400"("Bad request format") 이어야 한다.

데이터 형식은 boolean 이어야 한다.

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "all_confirmation"의 예:

```
true
```

7.7 capabilities

능력 "capabilities"는 각각 호스트가 지원하는 "capability"를 나타내는 문자열의 배열이어야 한다.

호스트가 선택 사항인 동기 모드를 지원하는 경우 이 능력을 "synchronous_mode"로 보고하는 것이 좋다. 클라이언트는 동기 모드 요청을 보내기 전에 이 능력을 확인하는 것이 좋다.

데이터 형식은 array 여야 한다.

Maximum items: 1 000

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "capabilities"의 예:

```
[  
  "synchronous_mode"  
]
```

7.8 capability

능력 "capability"는 호스트가 지원하는 능력을 나타내는 문자열이어야 한다.

데이터 형식은 string 이어야 한다.

Maximum length: 128

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화된 "capability"의 예:
"synchronous_mode"

7.9 ext_key_container

확장 키 요청 컨테이너 "ext_key_container"는 "/kmapi/v1/ext_keys" 엔드포인트 호출에 사용되는 컨테이너이다.

데이터 형식은 <표 7.9-1>에 규정된 대로 IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

<표 7.9-1> "ext_key_container"의 데이터 유형

Field Name	Type	설명
keys	array of object schema: keys	REQUIRED 키 "keys"는 "key"의 배열이어야 한다.
initiator_sae_id	string schema: initiator_sae_id	REQUIRED 개시 SAE ID "initiator_sae_id"는 하나 이상의 대상 SAE 와 키 공유를 최초로 요청한 SAE 의 ID 여야 한다. "ext_keys"에서 키를 참조하는 데 사용할 수 있다. 또한 "ext_keys/void"나 "ext_keys/ack"에서 이전 "ext_keys" 호출(들)의 키를 참조하는 데에도 사용할 수 있다. KME 는 (내부) SAE 로 동작하여, 역시 SAE 로 동작하는 다른 KME 와 키 공유를 요청할 수 있다. 요청의 출처에 개시 SAE 와 대상 SAE 의 구별이 적용되지 않는 경우, 구현은 임의로 하나를 "initiator_sae_id"로, 나머지를 "target_sae_id"로 지정할 수 있다. SAE ID 는 본문 4.6 절의 요구사항을 따라야 한다. Maximum length: 64
target_sae_ids	array of string schema: target_sae_ids	REQUIRED "target_sae_ids"는 요청과 관련된 대상 SAE 의 ID 인 "target_sae_id"의 배열이어야 한다. 대상은 하나 또는 여러 개를 지정할 수 있다. 개시 SAE 와 더불어 각 대상 SAE 는 키의 동일한 사본을 받을 자격이 있는 SAE 이다. KME 는 대상 SAE 를 하나만 지원(배열에 단일 SAE ID)하거나 지원하는 대상 SAE 의 최대 수를 제한할 수 있다. 가용 KME 의 능력을 초과하는 요청은 "503"('Error of server side') 응답 코드로 거부될 수 있다.
ack_callback_url	string schema: ack_callback_url	확인 응답 콜백 URL "ack_callback_url"은 요청의 전부 또는 일부가 완료되거나 실패한 후 확인 응답(들)을 보낼 URL 이다. 필수인 비동기 모드에서는 이 파라미터가 필수이며 지정되어야 한다. 이 파라미터가 생략되면 요청은 선택 사항인 동기 모드로 엔드포인트를 호출하는 요청으로 해석되어야 한다. 그렇지 않으면 필수인 비동기 모드로 엔드포인트를 호출하는 요청으로 해석되어야 한다. 필수 비동기 모드와 선택 동기 모드의 동작은 4.2 절을 참조한다. 새로운 KME 에 "ack_callback_url" 없이 동기 모드 요청을 보내기 전에, "/kmapi/versions" 엔드포인트를 호출하여 새 KME 가 선택 사항인 동기 모드 "capability"를 지원하는지 확인하는 것이 좋다. "ack_callback_url"을 제공하는 경우, 프로토콜로 시작하는, 즉 "https://"로 시작하는 완전한 URL 이어야 한다. authority 에는 도메인이나 IP 주소 외에 선택적으로 포트 번호를 포함할 수 있다. Format: uri Maximum length: 1 024
extension_mandatory	object schema: extension_mandatory	확장 "extension_mandatory"는 각각 ETSI 또는 다른 조직(예: 벤더)이 규정한 확장을 나타내는 임의 개수의 속성을 담은 객체여야 하며, KME 는 이를 처리하거나, "unsupported_mandatory_extension"이라는 "details" 항목을 추가하여 "503"('Error of server side response')으로 요청을 거부해야 한다.

Field Name	Type	설명
		수신 KME 가 "extension_mandatory" 확장의 규격에 있는 모든 요구사항을 인식하고 구현하지 않는 한, 요청을 거부해야 한다. KME 는 요청에 대해 구현했다고 주장하는 모든 확장의 규격을 따라야 한다. 관련되는 경우, 여기에는 확장의 규격에서 변경 불가(immutable)로 표시된 정보를 변경 없이 전달하는 것이 포함된다. 추가 요구사항은 4.5 절에 제시되어 있다. Minimum properties: 1 Maximum properties: 1 024
extension_optional	object schema: extension_optional	확장 "extension_optional"은 각각 ETSI 또는 다른 조직(예: 벤더)이 규정한 확장을 나타내는 임의 개수의 속성을 담은 객체여야 하며, KME 는 해당 확장을 지원하지 않더라도 이를 전달할 수 있다. 키 관리 개체가 "extension_optional" 확장의 규격에 있는 모든 요구사항을 인식하고 구현하지 않는 경우, 그 "extension_optional" 확장을 변경 없이 전달해야 한다. KME 는 요청에 대해 구현했다고 주장하는 모든 확장의 규격을 따라야 한다. 관련되는 경우, 여기에는 확장의 규격에서 변경 불가(immutable)로 표시된 정보를 변경 없이 전달하는 것이 포함된다. 추가 요구사항은 4.5 절에 제시되어 있다. Minimum properties: 1 Maximum properties: 1 024

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "ext_key_container"의 예:

```
{
  "keys": [
    {
      "key_id": "550e8400-e29b-41d4-a716-446655440000",
      "value": "wHHVxRwDJs3/bXd38GHP3oe4svTuRpZ50yCC7x4Ly\u002Bs=",
      "extension": {
        "E32473_extension1": "Some string applicable to a particular key",
        "E32473_extension2": {
          "property1": 10111,
          "property2": "Some text",
          "property3": {
            "subprop1": 21,
            "subprop2": true
          }
        }
      }
    }
  ],
  "initiator_sae_id": "encryptor1",
  "target_sae_ids": [
    "encryptor2"
  ],
  "ack_callback_url": "https://kme1/kmapi/v1/ext_keys/ack",
  "extension_mandatory": {
    "E32473_route_type": "direct",
    "E32473_method": {
      "hybrid_keys": true,
      "primary": "qkd",
      "secondary": "pqc"
    }
  },
  "extension_optional": {
    "E32473_extension1": "Some string applicable to all keys in the container",
    "E32473_module_type": {
      "vendor": "Company ABC",
      "protocol": "BB84",
      "min_version": 2.5
    },
    "E32473_qos_session": "e73d9abe"
  }
}
```

```
}  
}
```

7.10 extension

확장 "extension"은 각각 ETSI 또는 다른 조직(예: 벤더)이 규정한 확장을 나타내는 임의 개수의 속성을 담은 객체여야 한다. KME 는 일련의 확장을 구현하도록 선택할 수 있다. 확장의 규격에는 요구사항이 포함될 수 있으며, 확장을 구현하는 KME 는 그러한 요구사항을 준수해야 한다.

추가 요구사항은 4.5 절에 제시되어 있다.

데이터 형식은 object 여야 한다.

Minimum properties: 1

Maximum properties: 1 024

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "extension"의 예:

```
{  
  "E32473_extension1": "Some string",  
  "E32473_extension2": {  
    "property1": 10111,  
    "property2": "Some text",  
    "property3": {  
      "subprop1": 21,  
      "subprop2": true  
    }  
  }  
}
```

7.11 extension_mandatory

확장 "extension_mandatory"는 각각 ETSI 또는 다른 조직(예: 벤더)이 규정한 확장을 나타내는 임의 개수의 속성을 담은 객체여야 하며, KME 는 이를 처리하거나, "unsupported_mandatory_extension"이라는 "details" 항목을 추가하여 "503"("Error of server side response")으로 요청을 거부해야 한다.

수신 KME 가 "extension_mandatory" 확장의 규격에 있는 모든 요구사항을 인식하고 구현하지 않는 한, 요청을 거부해야 한다.

KME 는 요청에 대해 구현했다고 주장하는 모든 확장의 규격을 따라야 한다. 관련되는 경우, 여기에는 확장의 규격에서 변경 불가(immutable)로 표시된 정보를 변경 없이 전달하는 것이 포함된다.

추가 요구사항은 4.5 절에 제시되어 있다.

데이터 형식은 object 여야 한다.

Minimum properties: 1

Maximum properties: 1 024

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "extension_mandatory"의 예:

```
{  
  "E32473_module_type": {  
    "vendor": "Company ABC",  
    "protocol": "BB84",  
    "min_version": 2.5  
  },  
  "E32473_qos_session": "e73d9abe"  
}
```

7.12 extension_optional

확장 "extension_optional"은 각각 ETSI 또는 다른 조직(예: 벤더)이 규정한 확장을 나타내는 임의 개수의 속성을 담은 객체여야 하며, KME 는 해당 확장을 지원하지 않더라도 이를 전달할 수 있다.

키 관리 개체가 "extension_optional" 확장의 규격에 있는 모든 요구사항을 인식하고 구현하지 않는 경우, 그 "extension_optional" 확장을 변경 없이 전달해야 한다.

KME 는 요청에 대해 구현했다고 주장하는 모든 확장의 규격을 따라야 한다. 관련되는 경우, 여기에는 확장의 규격에서 변경 불가(immutable)로 표시된 정보를 변경 없이 전달하는 것이 포함된다.

추가 요구사항은 4.5 절에 제시되어 있다.

데이터 형식은 object 여야 한다.

Minimum properties: 1

Maximum properties: 1 024

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "extension_optional"의 예:

```
{
  "E32473_module_type": {
    "vendor": "Company ABC",
    "protocol": "BB84",
    "min_version": 2.5
  },
  "E32473_qos_session": "e73d9abe"
}
```

7.13 initiator_sae_id

개시 SAE ID "initiator_sae_id"는 하나 이상의 대상 SAE 와 키 공유를 최초로 요청한 SAE 의 ID 여야 한다. "ext_keys"에서 키를 참조하는 데 사용할 수 있다. 또한 "ext_keys/void"나 "ext_keys/ack"에서 이전 "ext_keys" 호출(들)의 키를 참조하는 데에도 사용할 수 있다.

KME 는 (내부) SAE 로 동작하여, 역시 SAE 로 동작하는 다른 KME 와 키 공유를 요청할 수 있다. 요청의 출처에 개시 SAE 와 대상 SAE 의 구별이 적용되지 않는 경우, 구현은 임의로 하나를 "initiator_sae_id"로, 나머지를 "target_sae_id"로 지정할 수 있다.

SAE ID 는 본문 4.6 절의 요구사항을 따라야 한다.

데이터 형식은 string 이어야 한다.

Minimum length: 1

Maximum length: 64

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "initiator_sae_id"의 예:

```
"encryptor1"
```

7.14 key_id

키 ID "key_id"는 키의 ID 이며 IETF RFC 9562 [8]의 4 절에 규정된 UUID 여야 한다.

데이터 형식은 string 이어야 한다.

Format: uuid

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "key_id"의 예:

```
"550e8400-e29b-41d4-a716-446655440000"
```

7.15 key_id_container

키 ID 컨테이너 "key_id_container"는 객체의 배열이어야 하며, 각 객체는 "key_id"와 선택적인 "extension" 정보를 제공한다.

데이터 형식은 <표 7.15-1>에 규정된 객체의 배열로, IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

Maximum items: 1 024

<표 7.15-1> "key_id_container" 배열 요소의 데이터 유형

Field Name	Type	설명
key_id	string schema: key_id	REQUIRED 키 ID "key_id"는 키의 ID이며 IETF RFC 9562 [8]의 4 절에 규정된 UUID 여야 한다. Format: uuid
extension	object schema: extension	확장 "extension"은 각각 ETSI 또는 다른 조직(예: 벤더)이 규정한 확장을 나타내는 임의 개수의 속성을 담은 객체여야 한다. KME 는 일련의 확장을 구현하도록 선택할 수 있다. 확장의 규격에는 요구사항이 포함될 수 있으며, 확장을 구현하는 KME 는 그러한 요구사항을 준수해야 한다. 추가 요구사항은 4.5 절에 제시되어 있다. Minimum properties: 1 Maximum properties: 1 024

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "key_id_container"의 예:

```
[
  {
    "key_id": "550e8400-e29b-41d4-a716-446655440000",
    "extension": {
      "E32473_vendor": "Company ABC",
      "E32473_protocol": "BB84"
    }
  }
]
```

7.16 key_ids

키 ID "key_ids"는 "key_id"의 배열이어야 한다.

데이터 형식은 array 여야 한다.

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "key_ids"의 예:

```
[
  "550e8400-e29b-41d4-a716-446655440000",
  "373b0b2c-d841-4765-af6f-c6232cda6531",
  "65380d2d-7de4-4445-9a1e-23f77218e61d"
]
```

7.17 key

키 "key"는 "key_id"와 그 "value", 그리고 선택적인 "extension" 사전을 담은 객체여야 한다.

데이터 형식은 <표 7.17-1>에 규정된 대로 IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

<표 7.17-1> "key"의 데이터 유형

Field Name	Type	설명
key_id	string schema: key_id	REQUIRED 키 ID "key_id"는 키의 ID이며 IETF RFC 9562 [8]의 4 절에 규정된 UUID 여야 한다. Format: uuid
value	string schema: value	REQUIRED 값 "value"는 IETF RFC 4648 [7]에 규정된 base64 데이터 부호화 방식으로, 표 1의 알파벳을 사용하여 부호화된 키 데이터여야 한다. 구현은 base64 데이터 부호화 방식에서 사용되는 패딩이 절대 키 자료로 사용되지 않도록 보장해야 한다. 여기에는 부호화 입력의 마지막 단위(quantum)가 정확히 24 비트, 8 비트 또는 16 비트인 경우 출력의 마지막 부호화 단위 끝에 붙는 각각 0 개, 2 개 또는 1 개의 "=" 패딩 문자가 포함된다. 정수 바이트 크기가 아닌 키를 사용하는 경우, 복호화 시 (오른쪽에) 추가된 값 0의 패딩 비트를 반드시 제거해야 한다. 복호화된 키의 끝에서 값 0인 비트를 모두 제거하는 것은 키에 편향을 일으킬 수 있으므로 안전하지 않다. 유효한 키를 복원하려면 요청된 키 크기에 대한 별도의 정보를 이용해 이러한 패딩을 올바르게 제거해야 한다. (base64 데이터 부호화 방식과 그에 포함된 "=" 패딩 문자 규칙은 부호화 입력의 크기를 정수 바이트 단위로만 나타낼 수 있다. 정수 바이트 크기가 아닌 키의 경우, 부호화 출력의 마지막 문자 또는 첫 번째 "=" 패딩 문자 직전의 마지막 문자에는 부호화 중 추가된 값 0의 패딩 비트 정보가 포함될 수 있다.) 정수 바이트 크기가 아닌 키의 지원은 선택 사항이며, 벤더는 정수 바이트 크기만 지원하도록 선택할 수 있다.
extension	object schema: extension	확장 "extension"은 각각 ETSI 또는 다른 조직(예: 벤더)이 규정한 확장을 나타내는 임의의 개수의 속성을 담은 객체여야 한다. KME 는 일련의 확장을 구현하도록 선택할 수 있다. 확장의 규격에는 요구사항이 포함될 수 있으며, 확장을 구현하는 KME 는 그러한 요구사항을 준수해야 한다. 추가 요구사항은 4.5 절에 제시되어 있다. Minimum properties: 1 Maximum properties: 1 024

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "key"의 예:

```
{
  "key_id": "550e8400-e29b-41d4-a716-446655440000",
  "value": "wHHVxRwDJs3/bXd38GHP3oe4svTurpZ50yCC7x4Ly\\u002Bs=",
  "extension": {
    "E32473_protocol": "BB84"
  }
}
```

역자 주 – "표 1의 알파벳"은 IETF RFC 4648 [7]의 Table 1(표준 base64 알파벳)을 가리킨다.

7.18 keys

키 "keys"는 "key"의 배열이어야 한다.

데이터 형식은 array 여야 한다.

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "keys"의 예:

```
[
  {
    "key_id": "550e8400-e29b-41d4-a716-446655440000",
    "value": "wHHVxRwDJs3/bXd38GHP3oe4svTurpZ50yCC7x4Ly\\u002Bs=",
    "extension": {
      "E32473_protocol": "BB84"
    }
  }
]
```

7.19 problem_details

문제 세부 정보 "problem_details"는 IETF RFC 9457 [9] "Problem Details for HTTP APIs"의 요구사항을 충족하는 객체여야 한다.

데이터 형식은 <표 7.19-1>에 규정된 대로 IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

<표 7.19-1> "problem_details"의 데이터 유형

Field Name	Type	설명
type	string	REQUIRED 문제 유형을 식별하는 URI 참조. Format: uri Maximum length: 1 024
status	integer	이 문제 발생에 대해 원 서버가 생성한 HTTP 상태 코드. Format: int32 Minimum: 100 Maximum: 599
title	string	사람이 읽을 수 있는 문제 유형의 짧은 요약. Maximum length: 1 024
details	object	문제에 관한 세부 정보를 전달하기 위한 하나 또는 여러 개의 문자열 속성을 담은 객체. 일반적으로 각 문자열 속성의 이름은 주제를 식별하고, 문자열 값은 더 자세한 정보를 제공한다. "details"는 문자열 사전으로 역직렬화할 수 있다. Minimum properties: 1 Maximum properties: 1 024

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "problem_details"의 예:

```
{
  "type": "https://qkd.etsi.org/gs020-interop-kms/invalid-parameter",
  "status": 400,
  "title": "Invalid parameter format",
  "details": {
    "malformed_property": "Property xxxx_xxxx does not match the expected format",
    "invalid_character": "Property xxxx_xxxx contains an invalid character"
  }
}
```

7.20 target_sae_id

대상 SAE ID "target_sae_id"는 개시 SAE가 최초에 키 공유를 요청한 상대 SAE의 ID여야 한다. "ext_keys"에서 키를 참조하는 데 사용할 수 있다. 또한 "ext_keys/void"나 "ext_keys/ack"에서 이전 "ext_keys" 호출(들)의 키를 참조하는 데에도 사용할 수 있다.

KME는 (내부) SAE로 동작하여, 역시 SAE로 동작하는 다른 KME와 키 공유를 요청할 수 있다. 요청의 출처에 개시 SAE와 대상 SAE의 구별이 적용되지 않는 경우, 구현은 임의로 하나를 "initiator_sae_id"로, 나머지를 "target_sae_id"로 지정할 수 있다.

SAE ID는 본문 4.6 절의 요구사항을 따라야 한다.

데이터 형식은 string이어야 한다.

Minimum length: 1

Maximum length: 64

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "target_sae_id"의 예:
"encryptor2"

7.21 target_sae_ids

"target_sae_ids"는 요청과 관련된 대상 SAE의 ID인 "target_sae_id"의 배열이어야 한다. 대상은 하나 또는 여러 개를 지정할 수 있다. 개시 SAE와 더불어 각 대상 SAE는 키의 동일한 사본을 받을 자격이 있는 SAE이다.

KME는 대상 SAE를 하나만 지원(배열에 단일 SAE ID)하거나 지원하는 대상 SAE의 최대 수를 제한할 수 있다. 가용 KME의 능력을 초과하는 요청은 "503"('Error of server side') 응답 코드로 거부될 수 있다.

데이터 형식은 array여야 한다.

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "target_sae_ids"의 예:

```
[  
  "encryptor2",  
  "encryptor3"  
]
```

7.22 value

값 "value"는 IETF RFC 4648 [7]에 규정된 base64 데이터 부호화 방식으로, 표 1의 알파벳을 사용하여 부호화된 키 데이터여야 한다.

구현은 base64 데이터 부호화 방식에서 사용되는 패딩이 절대 키 자료로 사용되지 않도록 보장해야 한다. 여기에는 부호화 입력의 마지막 단위(quantum)가 정확히 24 비트, 8 비트 또는 16 비트인 경우 출력의 마지막 부호화 단위 끝에 붙는 각각 0개, 2개 또는 1개의 "=" 패딩 문자가 포함된다.

정수 바이트 크기가 아닌 키를 사용하는 경우, 복호화 시 (오른쪽에) 추가된 값 0의 패딩 비트를 반드시 제거해야 한다. 복호화된 키의 끝에서 값 0인 비트를 모두 제거하는 것은 키에 편향을 일으킬 수 있으므로 안전하지 않다. 유효한 키를 복원하려면 요청된 키 크기에 대한 별도의 정보를 이용해 이러한 패딩을 올바르게 제거해야 한다. (base64 데이터 부호화 방식과 그에 포함된 "=" 패딩 문자 규칙은 부호화 입력의 크기를 정수 바이트 단위로만 나타낼 수 있다. 정수 바이트 크기가 아닌 키의 경우, 부호화 출력의 마지막 문자 또는 첫 번째 "=" 패딩 문자 직전의 마지막 문자에는 부호화 중 추가된 값 0의 패딩 비트 정보가 포함될 수 있다.)

정수 바이트 크기가 아닌 키의 지원은 선택 사항이며, 벤더는 정수 바이트 크기만 지원하도록 선택할 수 있다.

데이터 형식은 string이어야 한다.

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "value"의 예:

```
"wHHVxRwDJs3/bXd38GHP3oe4svTuRpZ50yCC7x4Ly\u002Bs="
```

7.23 version

버전 "version"은 호스트가 지원하는 API의 버전을 나타내는 문자열이어야 한다.

데이터 형식은 string이어야 한다.

Maximum length: 16

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "version"의 예:

```
"v1"
```

7.24 version_container

버전 컨테이너 "version_container"는 호스트가 지원하는 "versions"와 "capabilities"를 선택적인 "extension" 정보와 함께 나타내는 객체여야 한다.

데이터 형식은 <표 7.24-1>에 규정된 대로 IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

<표 7.24-1> "version_container"의 데이터 유형

Field Name	Type	설명
versions	array of string schema: versions	REQUIRED 버전 "versions"는 지원하는 API 버전 "version"의 배열이어야 한다. Maximum items: 1 000
capabilities	array of string schema: capabilities	능력 "capabilities"는 각각 호스트가 지원하는 "capability"를 나타내는 문자열의 배열이어야 한다. 호스트가 선택 사항인 동기 모드를 지원하는 경우 이 능력을 "synchronous_mode"로 보고하는 것이 좋다. 클라이언트는 동기 모드 요청을 보내기 전에 이 능력을 확인하는 것이 좋다. Maximum items: 1 000
extension	object schema: extension	확장 "extension"은 각각 ETSI 또는 다른 조직(예: 벤더)이 규정한 확장을 나타내는 임의 개수의 속성을 담은 객체여야 한다. KME 는 일련의 확장을 구현하도록 선택할 수 있다. 확장의 규격에는 요구사항이 포함될 수 있으며, 확장을 구현하는 KME 는 그러한 요구사항을 준수해야 한다. 추가 요구사항은 4.5 절에 제시되어 있다. Minimum properties: 1 Maximum properties: 1 024

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "version_container"의 예:

```
{
  "versions": [
    "v1"
  ],
  "capabilities": [
    "synchronous_mode"
  ]
}
```

7.25 versions

버전 "versions"는 지원하는 API 버전 "version"의 배열이어야 한다.

데이터 형식은 array 여야 한다.

Maximum items: 1 000

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "versions"의 예:

```
[
  "v1"
]
```

7.26 void_container

무효화 요청 컨테이너 "void_container"는 키 무효화를 요청하기 위해 "ext_keys/void" 엔드포인트에서 사용되는 객체여야 한다.

데이터 형식은 <표 7.26-1>에 규정된 대로 IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화되어야 한다.

<표 7.26-1> "void_container"의 데이터 유형

Field Name	Type	설명
key_ids	array of string schema: key_ids	REQUIRED 키 ID "key_ids"는 "key_id"의 배열이어야 한다.
initiator_sae_id	string schema: initiator_sae_id	REQUIRED 개시 SAE ID "initiator_sae_id"는 하나 이상의 대상 SAE 와 키 공유를 최초로 요청한 SAE 의 ID 여야 한다. "ext_keys"에서 키를 참조하는 데 사용할 수 있다. 또한 "ext_keys/void"나 "ext_keys/ack"에서 이전 "ext_keys" 호출(들)의 키를 참조하는 데에도 사용할 수 있다. KME 는 (내부) SAE 로 동작하여, 역시 SAE 로 동작하는 다른 KME 와 키 공유를 요청할 수 있다. 요청의 출처에 개시 SAE 와 대상 SAE 의 구별이 적용되지 않는 경우, 구현은 임의로 하나를 "initiator_sae_id"로, 나머지를 "target_sae_id"로 지정할 수 있다. SAE ID 는 본문 4.6 절의 요구사항을 따라야 한다. Maximum length: 64
target_sae_ids	array of string schema: target_sae_ids	REQUIRED "target_sae_ids"는 요청과 관련된 대상 SAE 의 ID 인 "target_sae_id"의 배열이어야 한다. 대상은 하나 또는 여러 개를 지정할 수 있다. 개시 SAE 와 더불어 각 대상 SAE 는 키의 동일한 사본을 받을 자격이 있는 SAE 이다. KME 는 대상 SAE 를 하나만 지원(배열에 단일 SAE ID)하거나 지원하는 대상 SAE 의 최대 수를 제한할 수 있다. 가용 KME 의 능력을 초과하는 요청은 "503"('Error of server side') 응답 코드로 거부될 수 있다.
ack_callback_url	string schema: ack_callback_url	확인 응답 콜백 URL "ack_callback_url"은 요청의 전부 또는 일부가 완료되거나 실패한 후 확인 응답(들)을 보낼 URL 이다. 필수인 비동기 모드에서는 이 파라미터가 필수이며 지정되어야 한다. 이 파라미터가 생략되면 요청은 선택 사항인 동기 모드로 엔드포인트를 호출하는 요청으로 해석되어야 한다. 그렇지 않으면 필수인 비동기 모드로 엔드포인트를 호출하는 요청으로 해석되어야 한다. 필수 비동기 모드와 선택 동기 모드의 동작은 4.2 절을 참조한다. 새로운 KME 에 "ack_callback_url" 없이 동기 모드 요청을 보내기 전에, "/kmapi/versions" 엔드포인트를 호출하여 새 KME 가 선택 사항인 동기 모드 "capability"를 지원하는지 확인하는 것이 좋다. "ack_callback_url"을 제공하는 경우, 프로토콜로 시작하는, 즉 "https://"로 시작하는 완전한 URL 이어야 한다. authority 에는 도메인이나 IP 주소 외에 선택적으로 포트 번호를 포함할 수 있다. Format: uri Maximum length: 1 024
extension	object schema: extension	확장 "extension"은 각각 ETSI 또는 다른 조직(예: 벤더)이 규정한 확장을 나타내는 임의 개수의 속성을 담은 객체여야 한다. KME 는 일련의 확장을 구현하도록 선택할 수 있다. 확장의 규격에는 요구사항이 포함될 수 있으며, 확장을 구현하는 KME 는 그러한 요구사항을 준수해야 한다. 추가 요구사항은 4.5 절에 제시되어 있다. Minimum properties: 1 Maximum properties: 1 024
all_confirmation	boolean schema: all_confirmation	"all_confirmation"은 확인 플래그 역할을 하며, "ext_keys/void" 호출에서 키 ID 가 지정되지 않았을 때 지정된 SAE 간의 모든 키를 무효화하려는 의도임을 확인하기 위해 "true" 값과 함께 포함되어야 한다. "ext_keys/void" 호출에서 키 ID 가 지정되지 않았는데 "all_confirmation"이 생략되었거나 그 값이 "true"가 아니면, 응답은 "details" 안에 "no_all_confirmation"이라는 문자열을 포함한 "400"('Bad request format') 이어야 한다.

예: IETF RFC 8259 [2]에 규정된 JSON 형식의 미디어 유형 "application/json"으로 부호화한 "void_container"의 예:

```
{
```

```

    "key_ids": [
      "550e8400-e29b-41d4-a716-446655440000",
      "373b0b2c-d841-4765-af6f-c6232cda6531",
      "65380d2d-7de4-4445-9a1e-23f77218e61d"
    ],
    "initiator_sae_id": "encryptor1",
    "target_sae_ids": [
      "encryptor2"
    ],
    "ack_callback_url": "https://kme1/kmapi/v1/ext_keys/ack",
    "extension": {
      "E32473_extension1": "Some string",
      "E32473_extension2": {
        "property1": 10111,
        "property2": "Some text",
        "property3": {
          "subprop1": 21,
          "subprop2": true
        }
      }
    }
  }
}

```

부속서 A (규범):

OpenAPI 표현

이 문서에 기술된 API의 OpenAPI 표현 최신 버전은 다음 URL에서 이용할 수 있다.

- <https://forge.etsi.org/rep/qkd/gs020-interop-kms>

그리고 이 V1.1.1 발행본과 연관된 버전은 다음 위치에서 계속 이용할 수 있다.

- <https://forge.etsi.org/rep/qkd/gs020-interop-kms/-/tree/v1.1.1>

이 OpenAPI 표현은 [10]에 규정된 "OpenAPI Specification v3.1.2"에 따라 해석할 때 이 문서의 규격과 동등한 것으로 간주할 수 있다. 이 문서는 대부분 OpenAPI 표현으로부터 생성되었다.

이력(History)

버전	일자	상태
V1.1.1	2026 년 6 월	발행